

Signal Processing Aspects of Computer Music: A Survey

JAMES ANDERSON MOORER

Invited Paper

Abstract—The application of modern digital signal processing techniques to the production and processing of musical sound gives the composer and musician a level of freedom and precision of control never before obtainable. This paper surveys the use of analysis of natural sounds for synthesis, the use of speech and vocoder techniques, methods of artificial reverberation, the use of discrete summation formulae for highly efficient synthesis, the concept of the all-digital recording studio, and discusses the role of special-purpose hardware in digital music synthesis, illustrated with two unique digital music synthesizers.

INTRODUCTION

THE FIELD of digital synthesis of musical sound is a fertile ground for the application of signal processing techniques. With the ever descending costs of computer technology, it is now becoming practical to rely entirely on digital means for sound production and processing. In this paper, we discuss some of the aspects of this particular marriage of art and technology, especially those currently in use at the Center for Computer Research in Music and Acoustics at Stanford University.

At first one might ask why the composer turns to the computer? The reason is first motivated by the drive for more freedom of expression. We wish not to be always limited by the constraints of the existing orchestral instruments. The second reason is precision. The computer is capable of "micro-surgery" on the sound waveform down to the level of the individual sample. The frequencies of digital oscillators, for instance, can be made as accurate and repeatable as need be. We are not so much interested in duplicating the sounds of existing instruments, although that is an important benchmark in verifying the validity of some of the synthesis techniques. We are more interested in modifications to and extensions of natural sounds, as well as completely new sounds. At each point in our research, however, we attempt to preserve as much "natural" quality as is possible, even with totally new timbres.

Let us first dissect the uses of the machine into somewhat artificial categories. The first kind of synthesis and probably the oldest kind is for the composer to sit down at a computer and write a program that produces the samples of the waveform of the sounds he wishes. We will call this "direct" synthesis. A variety of special-purpose languages have been developed toward this end (Mathews [42], Howe [29], Vercoe [73]). These languages usually provide a series of basic computational modules which are linked together ("cross patched") to produce the output waveform. A "piece" is

usually composed of two parts: An "instrument definition" which is a program for actually producing the sampled waveform of a particular sound, and an "event list" or "note list" which can be thought of as the "score" of the piece, providing, for instance, the starting times, durations, pitches, and loudness of the notes. Needless to say, many more options than these can be specified in the event list. Often the program defining the instrument can be arbitrarily complex. The point of going to a special purpose language rather than using a general purpose computer language for generating music is efficiency and conceptual ease of operation. The problem is that music is inherently a parallel process and not many easily available computer languages have this as an innate feature. Parallel processes must usually be simulated somehow, and special purpose languages can fill this need.

The second kind of synthesis is what we call "analysis-based" synthesis, where existing natural tones are digitized and analyzed. The results of the analysis are used as data to drive the synthesis. Recent progress in this field has made this concept somewhat more viable than was previously thought so that this promises to be a rich area of study in the future.

The third kind of synthesis is what we might term "musique concrète," to borrow a term from the tape music world. This is where we digitize natural sounds and use the computer to process them in various ways without actually analysing and resynthesizing as in the previous form. Except for the tremendous amount of data involved, this is probably the simplest form of synthesis. One advantage of using the computer over conventional tape studios is that the operations can be made noise-free by use of sufficient word length. Other advantages include precision and the availability of signal processing techniques that are not readily realized in analog form, such as changing the pitch of a sound without changing its duration.

We then can dissect the dimension of the kinds of processing we might wish to do in any of these various synthesis techniques. We often distinguish processing into "additive" techniques, "subtractive" techniques and "modulation" (non-linear) techniques, although it should be realized that these techniques are usually used in combination.

What we mean by "additive" synthesis is that a composite waveform is produced by the summation of component parts, for instance, adding up the harmonics of a tone to produce a single sound. This is sometimes called, somewhat misleadingly, "Fourier" synthesis. "Subtractive" synthesis is taking a spectrally rich waveform and carving away the undesired portions with filters. This is the model commonly used by the speech research community and is a popular technique in conventional analog music synthesizers. The term "subtractive" is a bit misleading, but since it is in common usage, we shall propagate it here.

In the last category, we dump everything that is not among the above categories. Possibly the most important part of the

Manuscript received September 16, 1976; revised November 29, 1976. This work was supported by a Grant from the National Endowment for the Arts and by NSF under Contracts DCR 75-00694 and BNS 75-17715.

The author is with the Center for Computer Research in Music and Acoustics, Department of Music, Stanford University, Stanford, CA 94305.

nonlinear techniques is a class we call summation formulas. These are closed-form expressions for various trigonometric summations. This gives us the ability to generate a whole wad of sinusoidal partials by evaluating a single expression of sometimes remarkable simplicity.

We can also classify synthesis *systems* by their turn-around time. This varies from *off-line* to *interactive* to *real-time*. The ultimate music synthesis system would be totally real-time, where the composer could adjust synthesis parameters dynamically, and could key in passages using an organ keyboard or any of a number of other real-time input devices. Since high quality music (and speech) synthesis requires a great amount of computation, this seems to be an unreachable goal without the help of special-purpose hardware. Two such special computing devices are described later in this paper.

Most of the music languages that have been developed to date are entirely off-line. This usually results when the computer is not dedicated entirely to music but is instead shared by many groups of competing interests. This means that the computation is done at one time and then the digital-to-analog conversion is done on the accumulated samples. This is a very difficult way to work! Since the criteria for quality are almost entirely subjective, a great deal of experimentation is usually required to get the desired sounds. Computer musicians are known as a heavy-lidded lot, in general, coming into local computation centers in the wee hours of the morning when they can get the machine to themselves so they can operate in an *interactive* mode. In this mode, we compute a short sound and play it back quickly, then immediately readjust the parameters and generate another experiment. This mode of operation can be quite fruitful and where real-time operation is not feasible; interactive operation should be encouraged. In as speculative a field as computer-generated music, the off-line mode of operation is almost unusable.

Most of the work done in the field of computer music to date has been unsupported. This being the case, it is no surprise that it is only now that the results of digital signal processing are being applied to music. One problem with funding is that there are no large-scale government agencies that can fund music work. In highly technical fields, any division of the military (for instance, the Advanced Research Projects Agency) as well as the National Science Foundation and National Aeronautics and Space Administration can be considered for funding. When research in the arts offers scientific contributions, then funding from more technical agencies like these can be justified. In the pure arts, there is only one government agency (National Endowment for the Arts), and a handful of private agencies (Ford and Rockefeller Foundations, for instance), most of which are more dedicated to support of the artist as an independent agent rather than support of large cooperative facilities and research. These agencies usually have strict policies against establishing facilities in general. If the particular project, however, has sufficient merit and credibility, exceptions can occasionally be made and some funding has been made available from these agencies.

Although it is possible that some of the results from musical research could have application to other areas of technology, it is somewhat unlikely because the goals are so different from common military and technical goals. This will become somewhat more clear in the following text. It is possible that some of the benefits of this research will have application in the commercial music field, but as yet, little or no research money from the commercial companies has been forthcoming.

Despite this somewhat dreary funding picture, applications of technology to the arts, and to music specifically, still

remain one of the most exciting and interesting fields, as judged by the amount of discussion in common popular technical literature. It seems as if everyone is interested in some form of music. Most homes have stereo equipment of some kind. We are surrounded by music everywhere we go. This being the case, we cannot help but think that there is good reason for research into the techniques of music production, and especially in the application of modern technology to one of the oldest of human activities.

WHAT IS MUSICAL SOUND?

It is certainly beyond the scope of this paper to try to define what music is, or even what musical sound is, but we can say a few things about some of the sounds that are generally considered to be musical in nature, such as the tones of common orchestral instruments and the human voice when singing.

We can classify musical sounds into *quasi-periodic* (or *harmonic*) and *aperiodic* (or *inharmonic*) although (as usual) few sounds are perfectly periodic or perfectly aperiodic. Most orchestral instruments are indeed quasi-periodic. This means that they can be described by a sum of a finite number of sinusoids. To account for the variation from perfect periodicity, we must allow both the amplitude and frequency of each harmonic to vary independently. Without this separate control, we cannot guarantee that the quasi-periodic nature of so many orchestral tones can be simulated (Grey [25]).

Figs. 1-4 show waveforms and spectra for four different orchestral instruments: the cello, the clarinet, the trumpet, and the bass drum. The first three are harmonic and the last is inharmonic. The upper plot is the waveform of an entire isolated note (the blotches in the centers of the upper plots are artifacts of the Xerographic process used to produce these pictures and do not represent features of the waveforms themselves). The center plot of each of the figures shows a close-up of a 10-ms segment of the waveform taken from near the center of each note. The bottom plot shows the discrete Fourier transform of an 80-ms segment near the center of each note. The transforms show certain artifacts that should be ignored, such as 60- and 120-Hz contamination. Each segment was windowed with a Hamming window (Blackman and Tukey [11]).

The tone of many orchestral instruments may be divided for convenience into three parts: the "attack" portion, the "steady-state" portion and the "decay" portion. As with all of these categorizations, nothing falls clearly into one category or another. For instance, many instruments (violin with vibrato, bells) do not exhibit any steady-state portion. This division is made mostly to give us some terminology for talking about the transient regions at the beginnings and endings of tones. If ultimate quality is not necessary, then many tones can be easily modeled by splicing together an attack portion, a steady-state portion of appropriate length, and a decay portion. This approach is both concise in concept and simple in implementation and is quite popular when efficiency is a primary goal. When absolute fidelity is the goal, more complex models must be used.

In Figs. 1-3, the upper plots are divided into attack, steady-state, and decay portions. It is clear that this division is somewhat tenuous for the cello because the tone seems to be constantly changing throughout its duration. The drum tone (Fig. 4) shows no steady-state region either.

Most percussion instruments (drums, bells) are inharmonic. This means that to describe them with sinusoids often requires a large number of such. There are some synthesis techniques

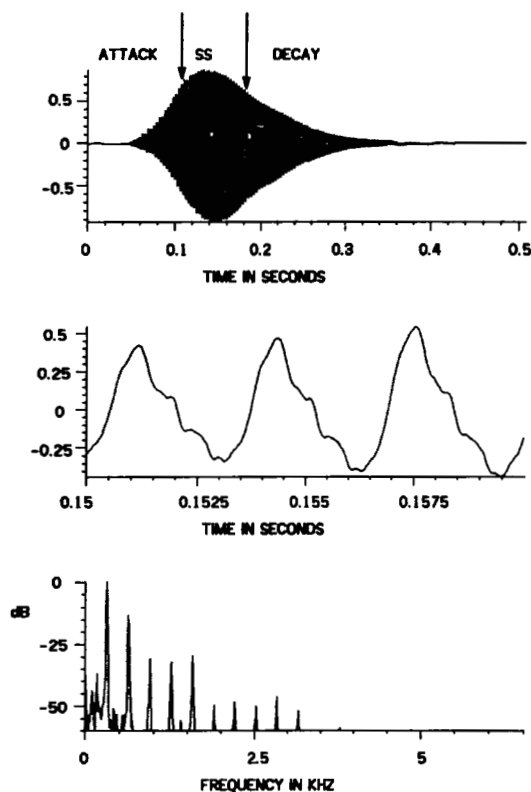


Fig. 1. Example of an isolated cello tone. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform containing about three periods. The lower plot is the discrete Fourier transform of an 80-ms segment in the middle of the note. The upper plot is divided, somewhat arbitrarily, into three segments: the *attack* portion, the *steady-state* portion, and the *decay* portion.

for creating what often turn out to be quite convincing drum-like or gong-like sounds, but to date, there are few analysis techniques that can be used with inharmonic sounds.

Let us look more closely at these figures. One striking feature of the clarinet tone (Fig. 2) is that there is a predominance of energy on the odd numbered harmonics. This distinction becomes obscured in the higher harmonics. Notice also that the clarinet waveform has a larger number of harmonics than either the trumpet or the cello. The clarinet attack is more rapid and the waveform is more complex than the others.

With the cello, we see an irregularity in the spectrum that is suggestive of resonances. In fact, the cello and orchestral strings in general have many resonances and can be modeled as an excitation (the bow and string) and a resonant system (the box) (Benade [8], [10] Mathews and Kohout [44]).

The trumpet tone exhibits a pulse-like waveform somewhat like that of voiced speech (Benade [9], [10]). Unlike voiced speech, however, the spectrum exhibits no formant behavior. The trumpet attack is faster than that of the cello, but the decay is slower than that of the clarinet. It is hard to see in the figure, but there is an irregularity in the attack portion of the trumpet.

We should make it clear that much of the characteristic sound of an instrument is in its transient regions, such as the attack portion of the tone. We have not shown the details of the attacks in these figures, but they differ greatly from one another. The attacks do seem to group themselves into families. There seems to be a characteristic clarinet-like

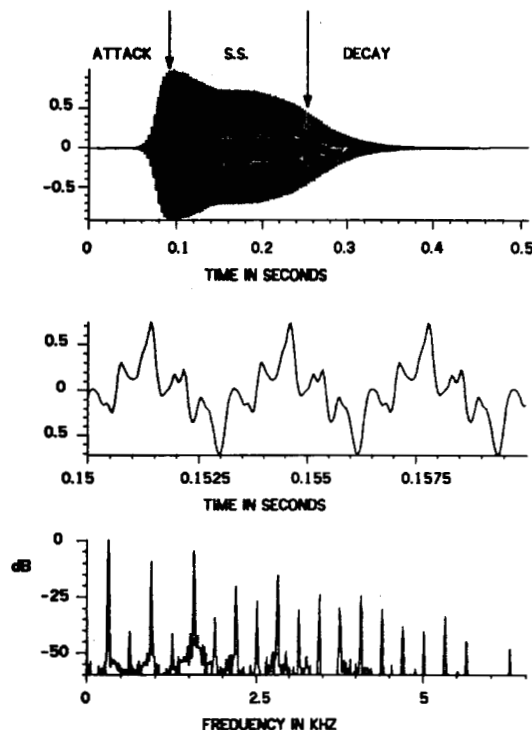


Fig. 2. Example of an isolated clarinet tone. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform containing about three periods. The lower plot is the discrete Fourier transform of an 80-ms segment in the middle of the note. The upper plot is divided, again somewhat arbitrarily, into three segments: the *attack* portion, the *steady-state* portion, and the *decay* portion. In comparing these plots with the previous plots for the cello tone, we can see the following differences: 1) The clarinet has more energy in the odd harmonics than the even harmonics, 2) The clarinet has a larger number of significant harmonics than the cello, and 3) The attack and decay of the clarinet are much faster than those of the cello.

attack, a characteristic brass-like attack, a string-like attack, and so on (Grey [25]).

Human speech exhibits both periodic and aperiodic aspects. Voiced speech is, in general, quite periodic. Many consonants (fricatives especially) are quite inharmonic. This provides a double problem, for any analysis technique to be applied to speech must be capable of dealing with harmonic and inharmonic signals.

MODELING, SYNTHESIS, AND OVERVIEW

Our goal in computer music is to produce a sampled-data signal which when converted to an analog voltage and played through a high-quality sound system produces the desired musical sound. The problem, then, is to synthesize this sampled-data signal. We do this by the use of various different *fundamental synthesis techniques*.

A synthesis technique is a formula for producing the next sample of a sampled-data signal. Probably the simplest example of this would be the following:

$$X(n) = \begin{cases} \sin(\omega_0 nT), & n_0 \leq n < n_1 \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where $X(n)$ is the signal at time nT , n is the sample number (time index), T is the time between consecutive samples, and ω_0 is the radian fundamental frequency of the note.

This says that the output waveform will be a sinusoid of frequency ω_0 that begins at sample n_0 and continues until

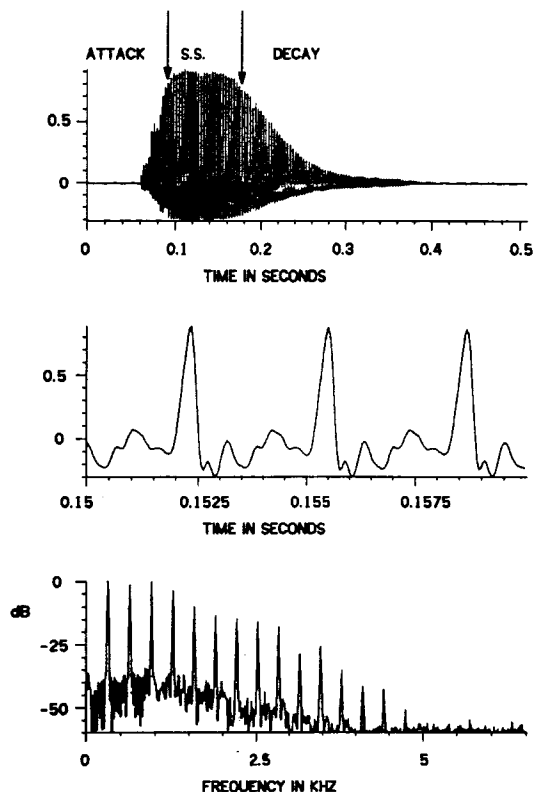


Fig. 3. Example of an isolated trumpet tone. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform containing about three periods. The lower plot is the discrete Fourier transform of an 80-ms segment in the middle of the note. The upper plot is divided into three segments: the *attack* portion, the *steady-state* portion, and the *decay* portion.

sample n_1 . This produces a single note at sample n_0 , of duration $(n_1 - n_0)$ samples, and frequency ω_0 . By combining many of these notes together, we can produce melodies or even entire pieces.

The problem with this particular synthesis technique is that the sound is not terribly exciting. To help us decide what to do about this, we can look at the waveforms produced by existing musical instruments for hints. We see immediately that most music instruments have much more complex spectra, and that the signal changes over its duration. The next simplest thing we can do is to multiply our sinusoid by some time-varying scale factor so that amplitude of the signal starts at zero, rises to a maximum in some amount of time (20 ms for example), persists at this maximum, then decays to zero. Another thing we might do is to use more and more sinusoidal signals in this waveform to enhance the spectrum.

We see then that a synthesis technique is a way to produce a sampled-data signal that generally can be expressed as the evaluation of a mathematical expression for the signal. This expression usually has several variables, such as pitch, duration, and loudness. Often we want these variables to change with time over the duration of the sound. This means that these variables must be functions of time. Fig. 5 diagrams this process. It is generally assumed that these controlling functions of time change much more slowly than the actual output waveform so that they can be sampled at a much lower rate.

In what follows, we will describe several different synthesis techniques. In each synthesis technique, we can identify the synthesis expression which must be evaluated for each output sample, and what the controlling functions are. The tech-

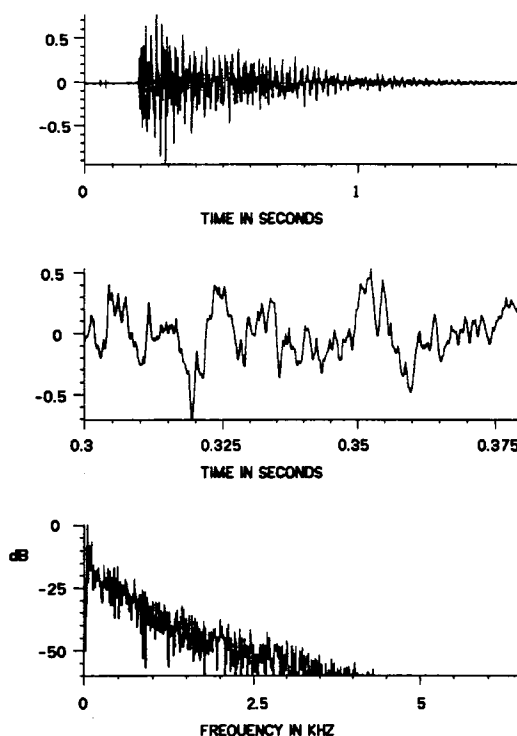


Fig. 4. Example of an isolated bass drum note. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform. The lower plot is the discrete Fourier transform of a 160-ms segment in the middle of the note. 160 ms was used instead of 80 ms because the tone contained frequencies much lower than those of the other tones.

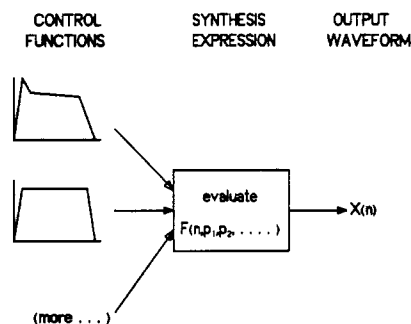


Fig. 5. General form of a fundamental synthesis technique. The object is to produce a sampled-data waveform as output with the desired sound quality. This is usually done by evaluating a mathematical expression (function) that produces the next sample of the output waveform. This expression usually has several variables which must be assigned values, such as pitch, amplitude, and duration. Often we wish these values to change slowly over the course of the tone. To do this, we use a series of control functions, one for each parameter we wish to vary. In analysis-based synthesis, we get these control functions from the analysis of natural sounds. In direct synthesis, these control functions are entered by the composer, usually in piecewise-linear form. One set of control functions will produce one "note." To assemble an entire piece, we may add or concatenate in time the results of many different fundamental synthesis techniques with different sets of control parameters. The expression in the box, $F(n, p_1, p_2, \dots)$ was just meant to indicate some mathematical expression that is a function of n , the sample number, and some set of parameters p_1, p_2 , and so on. These parameters may be either single numbers or functions of time (control functions). Several examples of this expression F are given in the text.

niques differ in what the synthesis expression is, what parameters must be controlled and what they do, and on the origin of the control functions. To illustrate this last point, in *direct*

synthesis, the controlling functions are entered by hand by the composer, usually as piecewise-linear functions. In *analysis-based* synthesis, the functions are made from processing sampled-data waveforms of the sounds of existing music instruments. They can then be modified by the composer. In *musique concrète*, the controlling functions are unmodified digitized sounds.

The synthesis expression is a mathematical *model* of the waveform. In analysis-based synthesis, the analysis procedure is an attempt to *estimate* the parameters of the model for a given sound. The extent to which the synthesized sound matches the original sound is a measure of the validity of the model and the estimation technique. If the model is not valid for the sound in question, or if the estimation technique is not accurate, we cannot hope to duplicate the original sound.

ANALYSIS-BASED ADDITIVE SYNTHESIS

Let us begin further explanation of the details with an analysis technique for use with the additive synthesis model. Using this model, we represent the signal as follows:

$$X(n) = \sum_{k=1}^M A_k(n) \sin \{nT[k\omega + 2\pi F_k(n)]\} \quad (2)$$

where $X(n)$ is the signal at time nT , n is the sample number (time index), T is the time between consecutive samples, ω is the radian fundamental frequency of the note, k is the harmonic number, $A_k(n)$ is the amplitude of harmonic k at time nT (it is assumed to be *slowly* time varying), M is the number of harmonics, and $F_k(n)$ is the frequency deviation of harmonic k at time nT (it is assumed to be *slowly* time varying).

Fig. 6 shows this model. The waveform is represented here by a sum of harmonically related sinusoids with slowly time-varying amplitudes and frequencies. It is then the purpose of the analysis technique to estimate these amplitudes and frequencies independently for each harmonic. Equation (2) is both the *model* of the waveform and the *synthesis technique* for the class of harmonic signals. As a model of a waveform, we say that $X(n)$ is a sampled-data function that can be decomposed into M harmonics. As a synthesis technique, we say that $X(n)$ represents the n th output sample of a waveform being synthesized, and that waveform can be computed as the sum of M sinusoids. Specifying values for the various numbers and functions involved (ω , M , T , $A_k(n)$, and $F_k(n)$), specifies entirely the waveform for all time. In Fig. 6, we show these control functions controlling sinusoidal oscillators, the outputs of which are summed to produce the output waveform. The control functions shown in Fig. 6 were taken from the analysis of a clarinet tone.

We can either get these numbers and functions by making them up, or we can get them through analysis of existing musical sounds. In this section, we shall discuss a way of estimating these numbers using (2) as a *model* of the sound, and then using (2) as a *synthesis technique* to reproduce the sound, and then *modifying* the numbers to produce new sounds.

We should point out that we are engaging in a slight abuse of terminology here. The term "harmonics" technically refers only to sinusoidal components with frequencies that are *exactly* integral multiples of the fundamental frequency. The word "partials" refers to sinusoidal components with no constraints on the frequencies. We are using the term "har-

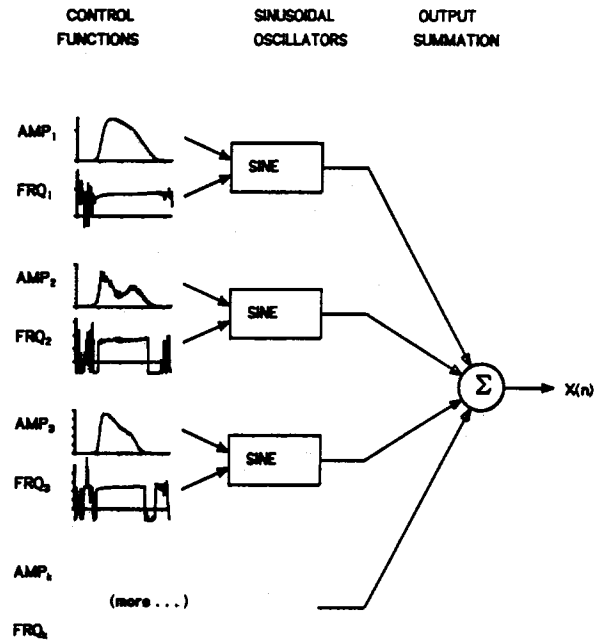


Fig. 6. General form of the additive synthesis technique. Here we control the amplitudes and frequencies of some number of sinusoidal oscillators and sum their outputs to produce the output signal.

monics" to refer to sinusoidal components with frequencies that are *nearly* integral multiples of some fundamental frequencies. They are not necessarily exactly harmonic.

Fig. 7 shows one form of an analysis technique that could be used to take a sampled-data function and analyse it in terms of this model. Here we see that the waveform is subjected to a bank of analysis devices, each one of which extracts one harmonic from the waveform and detects its frequency and amplitude as a function of time. The amplitude and frequency functions that are shown in Fig. 7 are the same as were in Fig. 6 and were derived from the analysis of a clarinet tone. One way of estimating these harmonic amplitudes and frequencies is as follows:

$$a_k(n) = \sum_{r=n}^{n+N-1} X(r) \sin(rk\omega_0 T) \quad (3)$$

$$b_k(n) = \sum_{r=n}^{n+N-1} X(r) \cos(rk\omega_0 T) \quad (4)$$

$$\hat{A}_k(n) = \sqrt{a_k^2(n) + b_k^2(n)} \quad (5)$$

$$\hat{\theta}_k(n) = \text{atan} \left(\frac{a_k(n)}{b_k(n)} \right) \quad (6)$$

$$\hat{F}_k(n) = \frac{1}{2\pi} \frac{d\hat{\theta}_k(n)}{dt} \quad (7)$$

where ω_0 is the radian frequency of analysis, k is the harmonic number, $X(r)$ is the input waveform at time rT , and N is the integer nearest the number of samples in one period of the input waveform. Note that ω_0 should be set to $2\pi/NT$. The hats on the amplitude, angle, and frequency variables (A , θ , F) indicate that these are *estimates* of the actual values and are not necessarily equal to the exact values that may have generated the tone.

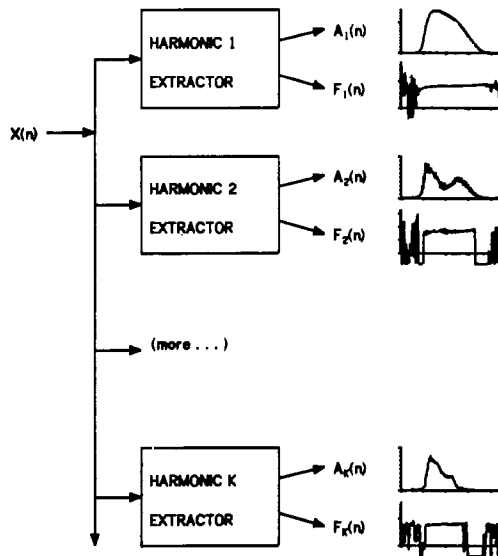


Fig. 7. General form of an estimator for additive synthesis technique. The object is to take a sampled-data waveform $X(n)$ of an isolated natural music instrument tone and to find the frequency and amplitude of each harmonic separately as functions of time.

In equation (7), we show a time derivative. Since the function we are differentiating is a sampled-data function, we must mention how this differentiation is to be accomplished. We recommend that a bandlimited linear-phase differentiator be used (Rabiner [59]).

Note that the pitch of the tone must be known, and it must be nearly constant over the interval of analysis. This technique is "pitch-synchronous." We rely on the summation over one period as in (3) and (4) to place a zero of transmission at the frequencies of the harmonics other than the one under analysis. A zero of transmission is a frequency where the sum shown in (3) and (4) is exactly zero, where the input signal is exactly annihilated. If the pitch is chosen incorrectly, then the frequencies of the zeros will not necessarily correspond to the frequencies of the adjacent harmonics. This means that the results will not correspond to a single harmonic, but to some kind of nonlinear combination of harmonics. Our goal with this technique is to extract each harmonic separately.

Empirical studies have been done by Moorer [50] by analysing synthetic signals of known harmonic content. These studies seem to indicate that this estimation method can tolerate no more than a 2-percent deviation (error) in the fundamental frequency of the input signal. This means that this method is only useful for *isolated* musical tones with *no vibrato*. There are recent developments in the phase vocoder which may allow these restrictions to be relaxed, but these results were not available in time for this publication (Moorer [51], Portnoff [58]).

The functions $A_k(n)$ and $F_k(n)$ are filtered by averaging over one period. The Z-transform of such a filter is $(1/N)(1 - Z^{-N})/(1 - Z^{-1})$. This filter places a zero of transmission at all harmonics except the one under analysis. This method is shown in Fig. 8 for the k th harmonic of a quasi-periodic signal. The input signal is first multiplied by a sine and cosine at the frequency of the harmonic in question. The resulting products are then averaged over one period of the fundamental frequency of the signal. The resulting sequences are then in phase quadrature. We can get the amplitude of the harmonic by taking the square root of the sum of the squares of the two

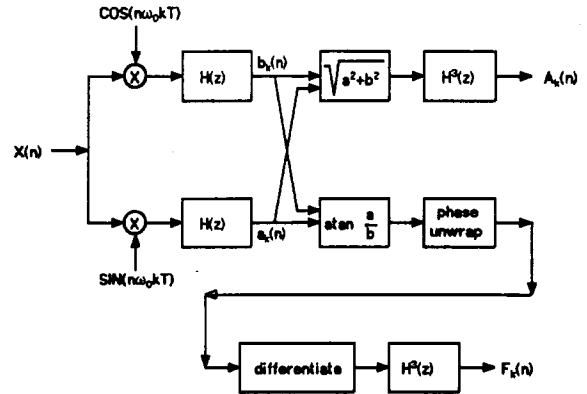


Fig. 8. Example of a harmonic extractor. The signal $X(n)$ is first multiplied by a sine and cosine at the frequency of the harmonic in question. Note that the period of the fundamental frequency ω_0 should be coerced to an integral number of samples N . That is, $\omega_0 \equiv 2\pi/N$. The transfer function $H(z)$ is just an average over one period (N points) and has the Z-transform $(1/N)(1 - Z^{-N})/(1 - Z^{-1})$.

sequences. We can get the frequency of the harmonic by first getting the angle between the two sequences as computed by the arctangent function. This produces the *principal value* of the phase angle. We must then regenerate the phase angle as a continuous function by "unwrapping" the principal value. Methods for doing this have been described in Schafer [64] and Moorer [50]. We can then differentiate the phase angle to get the frequency deviation. This deviation is the difference between the actual frequency of the harmonic and $k\omega_0$, the analysis frequency for that harmonic. These amplitude and frequency functions are then averaged over one period three more times to further cancel out the effects of adjacent harmonics on the analysis. Any "leakage" from adjacent harmonics will produce ripple on the estimated functions.

Although the technique as described by (3) through (7) is not linear, we can nonetheless get some idea of its behavior by examining the output when a pure sinusoid is analysed. Fig. 9 shows a plot of equation (5) for many values of frequency where $X(n) = \sin(2\pi f n T)$, a pure sinusoid. The analysis parameters in this case were $k = 5$, $\omega_0 = 2\pi 500$, $T = 1/25\ 600$, and $N = 1/(500T)$. This would be the amplitude of the fifth harmonic of a 500-Hz tone with a sampling rate of 25 600 Hz. We can see from the figure that there is indeed a zero of transmission at every harmonic except the fifth. This figure is the *amplitude* of the signal produced as the *estimate* of the fifth harmonic of a hypothetical 500-Hz tone when the input signal is, in fact, a pure sinusoid of unknown frequency between 0 Hz and 12 800 Hz.

If we apply more averaging (smoothing) such as is shown in Fig. 8, then the response to a pure sinusoid changes to that shown in Fig. 10. Here the "leakage" from the adjacent harmonics has been reduced greatly.

These figures are a bit misleading in that (5) describes a nonlinear process. This means that superposition does not really apply here and the concept of a frequency response is not meaningful. We can make a bit more sense of this by realizing that any frequency that is annihilated will not enter into the output regardless of the nonlinearity indicated by (5) and (6), and thus the importance of these figures is that they show where the greatest attenuation occurs.

To actually use this technique in practice, we perform the following procedure. We first digitize an isolated tone from a

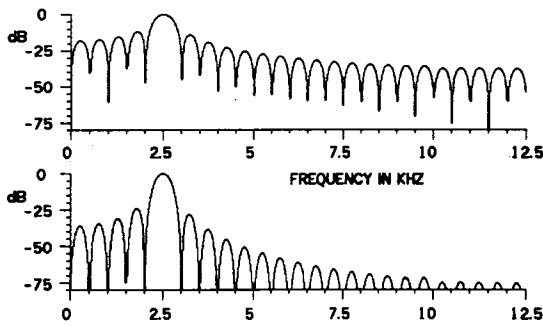


Fig. 9. Magnitude of equation (5) when the input is a pure sinusoid, evaluated for a wide range of frequencies. In this figure, we are analysing the fifth harmonic of a 500-Hz tone. In the lower plot, the output has been smoothed by averaging over one period of the fundamental of the analysis frequency.

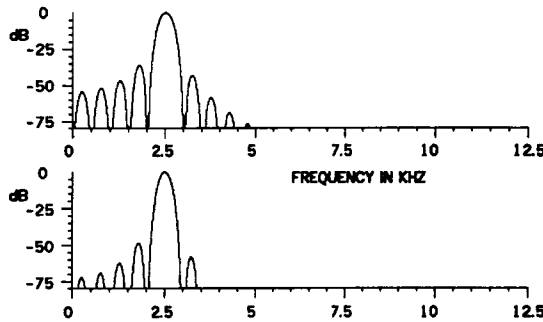


Fig. 10. Magnitude of equation (5) when the input is a pure sinusoid, as above, for the fifth harmonic of a 500-Hz tone and then smoothed by averaging over one period. The upper plot has been smoothed twice, and the lower plot has been smoothed three times. If we let N be the number of points in one period of the fundamental frequency, then the smoothing filter can be described by its Z -transform $(1/N)(1 - Z^{-N})/(1 - Z^{-1})$.

music instrument. We then determine its frequency as accurately as possible in any of a number of ways (Noll [52], Sondhi [71], Gold and Rabiner [22], Markel [37], Moorer [48], Miller [45]). Using this pitch information, we extract each harmonic separately, using the process shown in Fig. 8 which is partially specified by (3) through (7). This process must be applied M times, once for each harmonic that is present in the original waveform. In practice, we determine the value of M by applying the harmonic extractor to higher and higher harmonics until the output is no longer sensible, or until the amplitude detected is sufficiently small. An Algol program to do the computational portion of this procedure is given by Moorer [50].

This analysis data can be displayed in many ways. Two of the more intuitively appealing ways are shown in Fig. 11 and 12. This tone is an E^b above middle-C (about 311 Hz) played by a standard B^b clarinet. In the Fig. 11, just the amplitudes of the harmonics are displayed. Time goes left to right (the entire tone is about 0.4 s long), the first harmonic is in the rear, and the highest harmonic is in the fore. The vertical axis is linear. Fig. 12 shows a spectrograph-like representation where each harmonic is represented as a bar. The thickness of each bar is proportional to the log of the amplitude of that harmonic and the center of each bar is located at the frequency of that harmonic at that time.

As mentioned before and as diagrammed in Fig. 6, these data can be used directly to synthesize tones by use of (2). The first problem that one encounters in using this in an actual

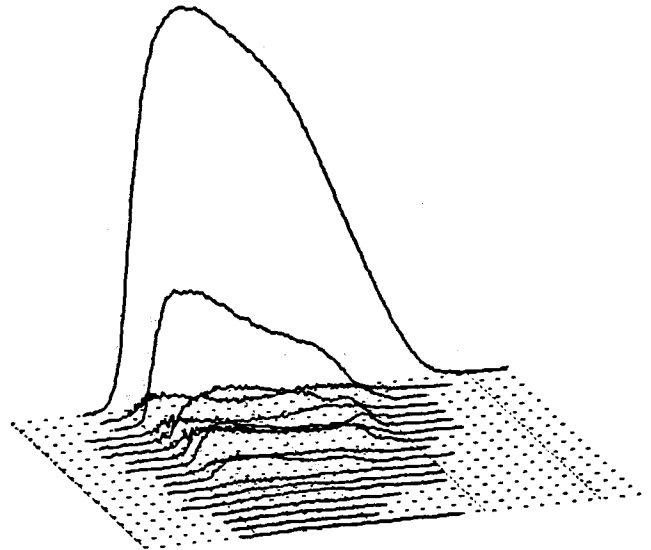


Fig. 11. Perspective plot of additive synthesis analysis data from a clarinet tone, shown as an amplitude $\times$ frequency $\times$ time perspective plot. The detailed frequency variation of each harmonic is not shown here. (X = time; Y = amplitude; Z = frequency, with the fundamental harmonic plotted in the background).

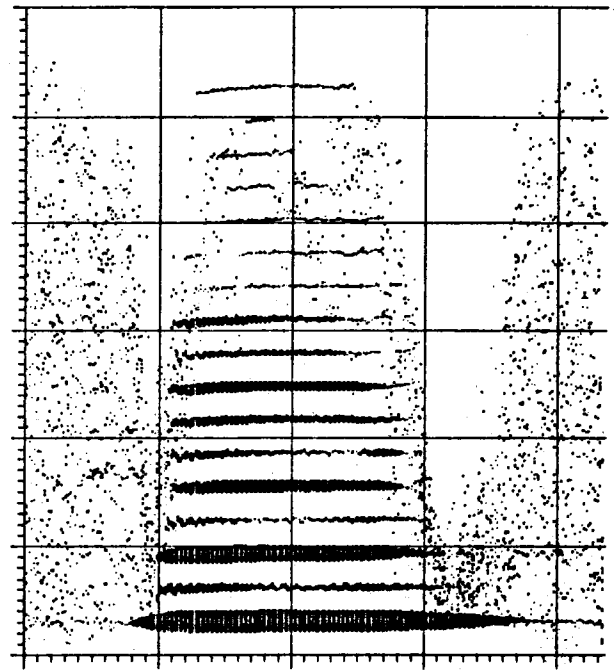


Fig. 12. Additive synthesis analysis data from the same clarinet tone as in Fig. 11, shown above in the form of a spectrographic plot (X = time, with 1/10 second lines; Y = frequency, with kHz lines; width of bars = relative dB to -40).

compositional context is the fact that these amplitude and frequency functions represent a tremendous amount of data. It is then of interest to find ways to reduce the amount of data involved. Data reduction is a common topic with the speech community (the vocoder problem, Oppenheim [54], Atal and Hanauer [4], Markel *et al.* [39], Markel and Gray [60], Makhoul *et al.* [34]) but the goals are different. In speech transmission, the goal is "adequate" fidelity with the lowest possible data rate through the channel. With musical sound, we wish the highest possible fidelity, and that is the first criterion.

There are many ways to reduce the data. For instance we could use Huffman coding or adaptive delta modulation or a number of other schemes, but these violate one of our criteria for the intermediate analysis data, which is that the analysis data must have an intuitively appealing form that can be easily modified by the composer. This requirement results from the fact that it is not our goal to create only replications of traditional sounds. There are excellent performers that do this quite well. The analysis data must be put in a form where the effect of changes on the resulting timbre is as apparent as possible while still preserving as much fidelity as is required. In the absence of changes, the analysis-synthesis path should be transparent. Small changes in the data should produce small changes in the resulting timbre. At each point, we judge the technique by how it sounds (perceptual criteria) and how easy it is to modify in predictable ways. Using these criteria rather than strictly information-theoretic criteria (for instance) results in greatly different intermediate representations.

One data reduction technique that has been quite successful is to fit each amplitude and frequency function with piecewise-linear functions (Grey [25]). This fit was done by hand with the aid of an interactive graphics program for viewing and modifying these sets of functions. Figs. 13 and 14 show the results of this kind of fit. In this manner, the data has been reduced from tens of thousands of numbers to around 200 numbers. A series of 16 instruments were analysed in this manner and the results were used as stimuli for a series of psychoacoustic experiments in (Grey [25]). These experiments were partly just to learn about our perception of timbre, but were also to test the perceptual validity of the analysis-synthesis system. The results were that the synthetic tones, even those represented by piecewise-linear functions, were judged to be very close to the originals. In no cases did the resulting tones sound synthetic or electronic. The differences were often described in terms of differences in articulation, rather than as differences between "real" and "unreal" sounds.

We can compare the amount of data reduction that the piecewise-linear fit gives us to the amount of data reduction obtained in some vocoder applications (Makhoul *et al.* [34]). In speech, the sampled waveform is usually digitized to 12 bits of accuracy at a sampling rate of 10000 Hz. This then represents 120 000 bits/s. By careful coding of the speech waveform, the data can be reduced to a much lower rate. If we choose 4800 bits/s as a median figure for reasonable quality, that represents a reduction of about 25 to 1. With the analysis and data reduction technique described above for musical tones, we usually digitize the waveform to 12 bits of accuracy at a sampling rate of 25 600 Hz, which would be 307 200 bits/s, and reduce the data to 32 functions (16 harmonics typically with 2 functions for each harmonic) with 7 line segments for each function. If these data are represented by 16-bit numbers, that represents about 3600 bits of data for an 0.5-s tone. This is a data reduction of about 43 to 1.

Thus, the data reduction ratio for this technique compares favorably with that obtained by vocoder techniques, but we must be careful to qualify the results by saying that the techniques have entirely different purposes. Again, with speech, we are interested in *adequate* quality for a wide range of possible input waveforms. The analysis technique as described above gives very high quality, but is only meaningful for *isolated* tones of *nearly-constant* frequency. We also require that the frequency be known *a priori*.

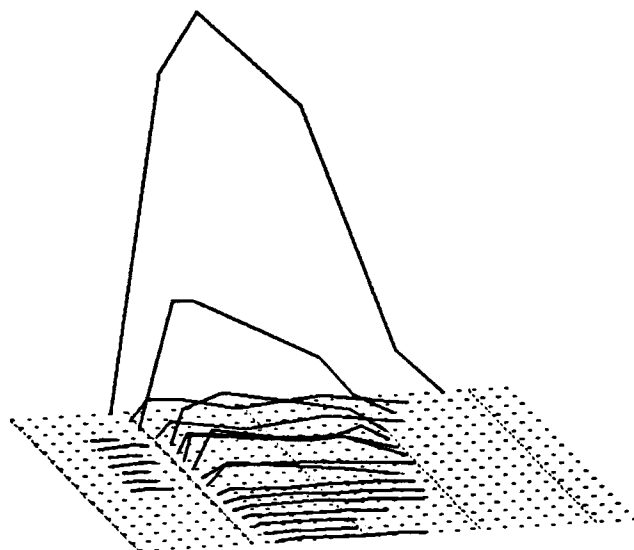


Fig. 13. Line segment approximations to the data shown in Figs. 11 and 12, shown here as an amplitude $\times$ frequency $\times$ time perspective plot (X = time; Y = amplitude; Z = frequency, with the fundamental harmonic plotted in the background).

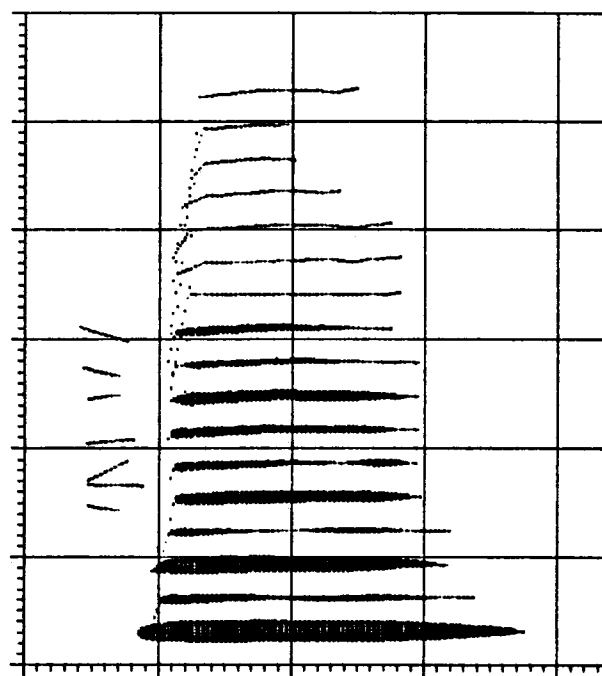
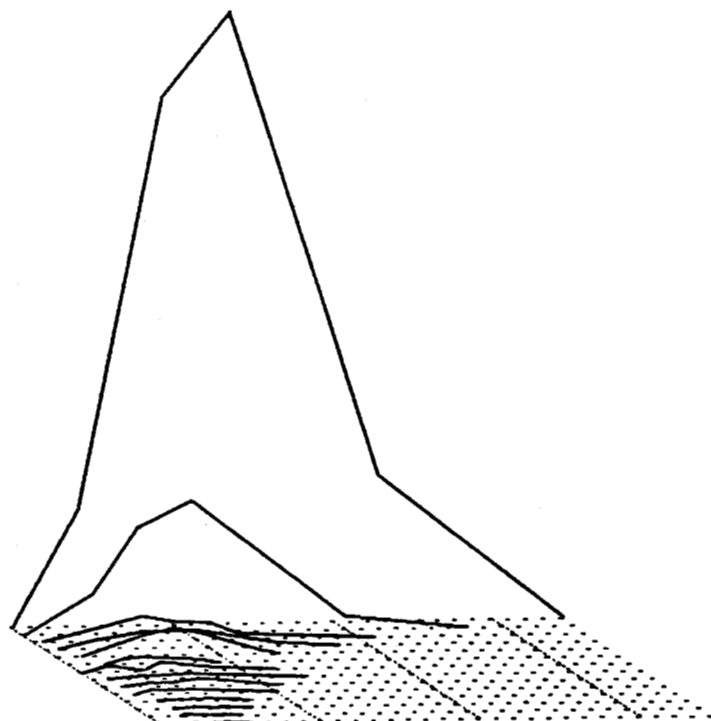


Fig. 14. Line segment approximations for synthesis, shown above in the form of a spectrographic plot (X = time, with 1/10 second lines; Y = frequency, with kHz lines; width of bars = relative dB to -40).

Figs. 15-17 show perspective plots of the piecewise-linear approximations to the amplitude functions of the cello tone, the clarinet tone, and the trumpet tone that were shown in Figs. 1-3. Also shown in the figures are the actual numbers used for the amplitude and frequency functions. The numbers are in ordered pairs. In each pair, the first number is the time in milliseconds of that breakpoint and the second number is the ordinate in hertz for the frequency function and in arbitrary amplitude units for the amplitude functions. This information is enough to synthesize these three tones.

These three tones have been equalized in pitch, amplitude, and duration by a perceptual matching experiment (Grey

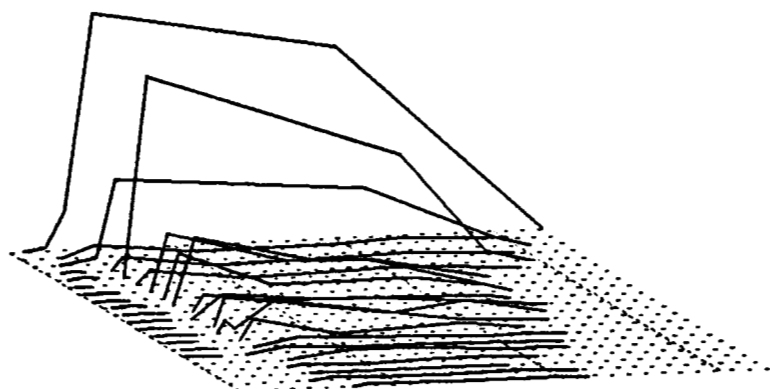


AMP 1	=	(1,0),(38,253),(81,1163),(122,1350),(224,317),(344,0)
FREQ 1	=	(1,95),(6,299),(43,314),(86,312),(274,313),(344,314)
AMP 2	=	(2,0),(41,82),(67,230),(101,287),(199,30),(277,0)
FREQ 2	=	(2,339),(9,605),(36,629),(119,627),(244,629),(277,626),(278,0)
AMP 3	=	(5,0),(66,53),(83,41),(106,37),(127,12),(212,0)
FREQ 3	=	(4,0),(5,554),(9,915),(29,938),(83,949),(183,950),(212,946),(213,0)
AMP 4	=	(8,0),(31,20),(58,36),(91,38),(141,10),(183,0)
FREQ 4	=	(6,0),(8,780),(13,1225),(40,1258),(115,1259),(167,1265),(183,1255),(184,0)
AMP 5	=	(15,0),(52,45),(72,60),(88,49),(126,8),(138,0)
FREQ 5	=	(13,0),(15,1342),(19,1589),(23,1554),(99,1583),(133,1581),(138,1501),(140,0)
AMP 6	=	(23,0),(33,11),(47,14),(67,12),(84,5),(94,0)
FREQ 6	=	(22,0),(23,1344),(31,1874),(62,1903),(72,1903),(81,1899),(94,1771),(95,0)
AMP 7	=	(2,0),(18,13),(38,1),(49,19),(79,8),(120,0)
FREQ 7	=	(2,1470),(6,2121),(20,2188),(74,2193),(101,2214),(120,2193),(122,0)
AMP 8	=	(9,0),(29,6),(63,10),(97,3),(117,3),(133,0)
FREQ 8	=	(8,0),(9,1655),(13,2459),(26,2513),(49,2510),(80,2530),(133,2525),(134,0)
AMP 9	=	(13,0),(27,7),(44,8),(61,16),(101,2),(109,0)
FREQ 9	=	(12,0),(13,2464),(15,2704),(22,2815),(72,2845),(95,2841),(109,2830),(110,0)
AMP 10	=	(12,0),(22,7),(55,10),(74,4),(94,4),(101,0)
FREQ 10	=	(11,0),(12,2483),(15,3107),(45,3151),(79,3169),(95,3169),(101,3158),(102,0)
AMP 11	=	(19,0),(23,2),(40,4),(56,4),(67,2),(74,0),(76,0)
FREQ 11	=	(18,0),(19,3131),(22,3463),(29,3438),(38,3459),(52,3467),(74,3451),(76,0)
AMP 12	=	(22,0),(29,2),(38,3),(51,2),(63,3),(72,0)
FREQ 12	=	(20,0),(22,3156),(24,3778),(29,3741),(34,3743),(51,3775),(72,3753),(73,0)
AMP 13	=	(19,0),(22,3),(24,4),(38,1),(52,3),(58,0)
FREQ 13	=	(18,0),(19,3278),(23,4078),(30,4063),(37,4087),(48,4092),(58,4079),(59,0)
AMP 14	=	(20,0),(24,1),(33,1),(40,2),(49,1),(56,0)
FREQ 14	=	(19,0),(20,3573),(22,4069),(29,4362),(40,4377),(47,4342),(56,4358),(58,0)

Fig. 15. Perspective plot and actual breakpoint information for the piecewise-linear approximation to the cello tone shown in Fig. 1. The data consists of 14 functions for the amplitudes of the harmonics and 14 functions for the frequencies of the harmonics. Each function is represented by its breakpoints, which are shown here as ordered pairs. The first number of each pair is the time in milliseconds of the breakpoint. The second number of each pair is the ordinate in Hertz for frequencies and arbitrary units for amplitudes. These data have been normalized so that this tone matches the clarinet and trumpet tones (Figs. 16 and 17) in pitch, duration, and loudness. The only perceptual difference is then the timbre.

[25]). The perspective plots in Figs. 15-17 are drawn to the same scale. The fact that these tones are perceived as being equally loud when there is such a strong disparity in the amplitudes of the partials comes as somewhat of a surprise to many people. These data provide an excellent base for testing theories of loudness perception (Moorer [49], [50]), pitch perception, and many more.

In fact, the analysis-based additive synthesis technique is a powerful tool for computer music and psychoacoustics. The piecewise-linear form gives a natural handle on modifying the tones. For example, by moving the breakpoints around at will we can produce a wide variety of new and unusual tones. One fascinating thing that can be done is to "interpolate" two different tones. Fig. 18 shows the functions for one instru-



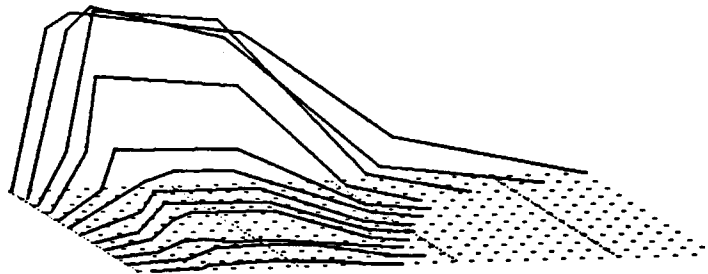
AMP 1 -	(7,0),(20,6),(32,73),(48,445),(199,361),(330,0)
FREQ 1 -	(6,0),(7,282),(19,368),(21,314),(46,310),(141,312),(284,313),(330,314)
AMP 2 -	(24,0),(43,22),(104,2),(193,4),(238,10),(301,0)
FREQ 2 -	(23,0),(24,629),(68,619),(116,616),(167,633),(223,624),(301,627),(302,0)
AMP 3 -	(15,0),(37,12),(48,159),(204,122),(286,17),(309,0)
FREQ 3 -	(14,0),(15,803),(24,928),(36,898),(46,931),(113,939),(330,942)
AMP 4 -	(9,0),(19,2),(24,0),(39,0),(46,26),(103,3),(167,5),(229,10),(291,0)
FREQ 4 -	(7,0),(9,1261),(24,1314),(30,327),(39,1245),(105,1243),(215,1257),(246,1249),(291,1261),(292,0)
AMP 5 -	(6,0),(18,5),(25,0),(39,0),(54,375),(212,210),(266,20),(295,0)
FREQ 5 -	(5,0),(6,1572),(25,1528),(32,911),(38,1560),(67,1554),(127,1565),(308,1569),(309,0)
AMP 6 -	(41,0),(48,25),(108,4),(216,12),(282,0)
FREQ 6 -	(2,0),(3,1934),(12,1890),(33,320),(46,1862),(186,1883),(282,1875),(283,0)
AMP 7 -	(2,0),(18,1),(24,0),(42,0),(52,108),(127,46),(177,42),(253,0)
FREQ 7 -	(1,0),(2,2180),(24,2128),(34,795),(43,2167),(113,2193),(253,2192),(255,0)
AMP 8 -	(2,0),(14,1),(23,0),(43,0),(52,83),(110,17),(199,18),(242,0)
FREQ 8 -	(1,0),(2,2458),(23,2343),(33,328),(45,2472),(125,2507),(242,2510),(243,0)
AMP 9 -	(5,0),(20,2),(21,3),(27,0),(42,0),(55,127),(132,73),(163,71),(255,0)
FREQ 9 -	(3,0),(5,2849),(27,2688),(33,964),(42,2792),(128,2822),(255,2819),(256,0)
AMP 10 -	(5,0),(23,1),(30,0),(47,0),(54,32),(92,17),(232,7),(247,0)
FREQ 10 -	(3,0),(5,3173),(30,3030),(39,2320),(50,3096),(134,3136),(247,3138),(248,0)
AMP 11 -	(1,0),(23,1),(28,0),(39,0),(59,44),(122,26),(153,26),(262,0)
FREQ 11 -	(1,3313),(28,3279),(34,1768),(43,3420),(127,3448),(262,3441),(264,0)
AMP 12 -	(1,0),(10,2),(21,0),(46,0),(52,53),(158,9),(206,28),(255,0)
FREQ 12 -	(1,3756),(21,3728),(33,2095),(47,3741),(136,3762),(255,3759),(256,0)
AMP 13 -	(3,0),(16,1),(29,0),(41,0),(46,24),(52,8),(77,57),(192,8),(250,0)
FREQ 13 -	(2,0),(3,4152),(29,3868),(36,2240),(46,4045),(85,4049),(128,4078),(181,4078),(250,4103),(251,0)
AMP 14 -	(3,0),(16,0),(20,0),(48,0),(56,38),(110,3),(188,14),(228,0)
FREQ 14 -	(2,0),(3,4213),(20,4119),(36,1566),(48,4344),(130,4388),(228,4388),(229,0)
AMP 15 -	(5,0),(23,1),(28,0),(50,0),(77,14),(202,1),(219,2),(247,0)
FREQ 15 -	(3,0),(5,4624),(28,4496),(33,1012),(48,4649),(122,4703),(247,4685),(248,0)
AMP 16 -	(14,0),(24,0),(38,0),(64,12),(104,4),(145,4),(215,1),(238,0)
FREQ 16 -	(1,4928),(24,4751),(36,1072),(52,4965),(117,5006),(155,5003),(188,5020),(238,3197),(239,0)
AMP 17 -	(58,0),(95,12),(136,13),(201,1),(220,3),(233,0)
FREQ 17 -	(45,0),(46,5005),(58,3759),(63,5285),(119,5325),(180,5325),(233,5367),(234,0)
AMP 18 -	(50,0),(61,5),(100,0),(141,4),(185,2),(208,0)
FREQ 18 -	(48,0),(50,4926),(52,5563),(94,5628),(113,5602),(137,5634),(208,5646),(210,0)
AMP 19 -	(58,0),(63,1),(85,0),(140,1),(171,0),(183,0)
FREQ 19 -	(56,0),(58,3938),(65,5753),(79,5930),(104,5889),(152,5916),(183,5880),(184,0)
AMP 20 -	(50,0),(64,5),(103,1),(139,1),(177,2),(219,0)
FREQ 20 -	(48,0),(50,5192),(58,6209),(121,6266),(190,6266),(204,6238),(219,6288),(220,0)
AMP 21 -	(70,0),(79,3),(113,3),(141,1),(206,1),(219,0)
FREQ 21 -	(69,0),(70,4245),(77,6537),(116,6567),(140,6571),(176,6564),(219,6583),(220,0)

Fig. 16. Perspective plot and actual breakpoint information for the piecewise-linear approximation to the clarinet tone shown in Fig. 2. The data consists of 21 functions for the amplitudes of the harmonics and 21 functions for the frequencies of the harmonics. Each function is represented by its breakpoints, which are shown here as ordered pairs. The first number of each pair is the time in milliseconds of the breakpoint. The second number of each pair is the ordinate in Hertz for frequencies and arbitrary units for amplitudes.

ment (the cello tone) slowly changing into the functions for another instrument (a French horn tone). If the instruments are somewhat similar to begin with, this produces a series of tones that indeed transform smoothly from one timbre to the other. If the instruments are very different, the intermediate tones often have very strange sounds, sometimes sounding like more than one instrument is playing.

ANALYSIS-BASED SUBTRACTIVE SYNTHESIS

In subtractive synthesis, we model the sound waveform as an excitation function sent through a time-varying filter. Fig. 19 shows a block diagram of this process. This technique has been used extensively to produce synthetic speech waveforms (Atal and Schroeder [3], Atal and Hanauer [4], Makhoul and Wolf [38], Makhoul [35], [36], Markel [37], Markel and



AMP 1 = (1,0),(20,305),(36,338),(141,288),(237,80),(360,0)
 FREQ 1 = (1,321),(16,324),(32,312),(109,310),(317,314),(360,310)
 AMP 2 = (3,0),(25,317),(39,361),(123,295),(222,40),(326,0)
 FREQ 2 = (2,0),(3,607),(16,657),(24,621),(133,621),(275,628),(326,628),(327,0)
 AMP 3 = (2,0),(19,100),(34,369),(111,342),(207,41),(273,0)
 FREQ 3 = (2,977),(5,782),(15,987),(24,932),(128,932),(217,936),(273,945),(275,0)
 AMP 4 = (1,0),(3,0),(24,113),(29,257),(118,231),(187,35),(235,0)
 FREQ 4 = (1,0),(2,0),(3,718),(16,1335),(24,1243),(108,1240),(199,1248),(235,1248),(236,0)
 AMP 5 = (1,0),(27,52),(34,130),(110,126),(191,13),(234,0)
 FREQ 5 = (1,1225),(9,1569),(12,1269),(21,1573),(37,1553),(97,1552),(181,1556),(234,1566),(235,0)
 AMP 6 = (1,0),(46,83),(64,100),(100,100),(189,11),(221,0)
 FREQ 6 = (1,1483),(12,1572),(23,1988),(33,1864),(114,1864),(177,1868),(221,1879),(222,0)
 AMP 7 = (1,0),(37,39),(45,77),(110,79),(176,11),(205,0),(207,0)
 FREQ 7 = (1,1792),(9,1612),(29,2242),(36,2174),(93,2176),(126,2170),(205,2188),(207,0)
 AMP 8 = (2,0),(28,17),(43,71),(109,66),(172,8),(201,0)
 FREQ 8 = (2,1590),(29,2539),(36,2491),(114,2481),(153,2489),(201,2491),(203,0)
 AMP 9 = (2,0),(29,16),(43,53),(54,66),(105,64),(165,7),(191,0)
 FREQ 9 = (2,1993),(25,2121),(32,2821),(37,2796),(84,2798),(105,2792),(191,2797),(192,0)
 AMP 10 = (1,0),(27,6),(41,25),(56,29),(72,22),(95,24),(180,0)
 FREQ 10 = (1,1742),(12,1849),(32,3131),(37,3111),(114,3103),(164,3116),(180,3116),(181,0)
 AMP 11 = (2,0),(37,6),(55,25),(88,29),(114,28),(164,3),(186,0)
 FREQ 11 = (2,1398),(41,3419),(42,3419),(91,3419),(106,3406),(150,3421),(186,3421),(187,0)
 AMP 12 = (7,0),(39,3),(43,8),(88,11),(118,9),(138,3),(165,0)
 FREQ 12 = (6,0),(7,1806),(23,2942),(36,2759),(37,3746),(50,3723),(84,3731),(110,3721),(156,3741),(165,3620),(167,0)

Fig. 17. Perspective plot and actual breakpoint information for the piecewise-linear approximation to the trumpet tone shown in Fig. 3. The data consists of 12 functions for the amplitudes of the harmonics and 12 functions for the frequencies of the harmonics. Each function is represented by its breakpoints, which are shown here as ordered pairs. The first number of each pair is the time in milliseconds of the breakpoint. The second number of each pair is the ordinate in Hertz for frequencies and arbitrary units for amplitudes. These data have been normalized so that this tone matches the cello and clarinet tones (Figs. 15 and 16) in pitch, duration, and loudness.

Gray [40], [41], Markel *et al.* [39]). For speech waveforms, the excitation is usually chosen to be either a periodic waveform such as a pulse train, or white noise. The periodic waveform is for use during voiced speech, such as vowels. The noise waveform is for use during unvoiced speech. The filter then simulates the resonances of the vocal tract. The filter must change with time, for the vocal tract resonances change with different positions of the tongue, lips, etc. This model can be represented by the following equation:

$$X(n) = - \sum_{k=1}^p a_k X(n-k) + G \sum_{r=0}^q b_r U(n-r) \quad (8)$$

a_k and b_r are the filter coefficients, by convention, b_0 is taken to be 1, $X(n)$ is the output waveform, G is the overall gain factor, and $U(n)$ is the input waveform, usually set to either a pulse train or white noise.

To synthesize some speech, then, we must determine at each point in time what kind of excitation to drive the filter with, and what kind of filter should be used. Choosing $U(n)$ and G determines the excitation, and choosing a_k and b_r determines the filter. This determination is usually done by analysis of actual speech waveforms. If we model the excitation by a pulse train or white noise, then we must determine the fundamental frequency of the required pulse train at each point in time, whether a pulse train or white noise is to be used (the

"voiced/unvoiced" decision), and the amplitude of the excitation. There are many ways to estimate these parameters. For instance, pitch detection is discussed in (Noll [52], Gold and Rabiner [22], Sondhi [71], Markel [38], Moorer [48], Miller [45]), voiced/unvoiced decision is discussed in (Makhoul *et al.* [34], Markel *et al.* [39], Atal and Rabiner [5]), amplitude determination is discussed in (Atal and Hanauer [4], Makhoul [35]).

A popular method of choosing the filter is described in Makhoul [35] which is based on computing a filter whose frequency response best matches the spectrum of the speech at each point in time in a minimum-squared-error sense. The most straightforward way of computing the filter coefficients assumes that the model is *all-pole*, that is, $b_r = 0$ for $r = 1, 2, \dots, q$. This is commonly called *linear predictive analysis*.

The method as described by Markel and Gray [40] has been used extensively by Dodge [19] to make a kind of music called "speech-song." The compositional process starts with reading poetry into a microphone which is connected to an analog-to-digital converter on a computer. The poetry is usually digitized in this manner in short segments. These digitized segments are then analysed using the autocorrelation method of linear prediction (Saito and Itakura [62], Itakura and Saito [30], Markel [38], Makhoul [35]). The analysis produces as intermediate data the voicing information (whether the periodic or noise excitation is to be used), the

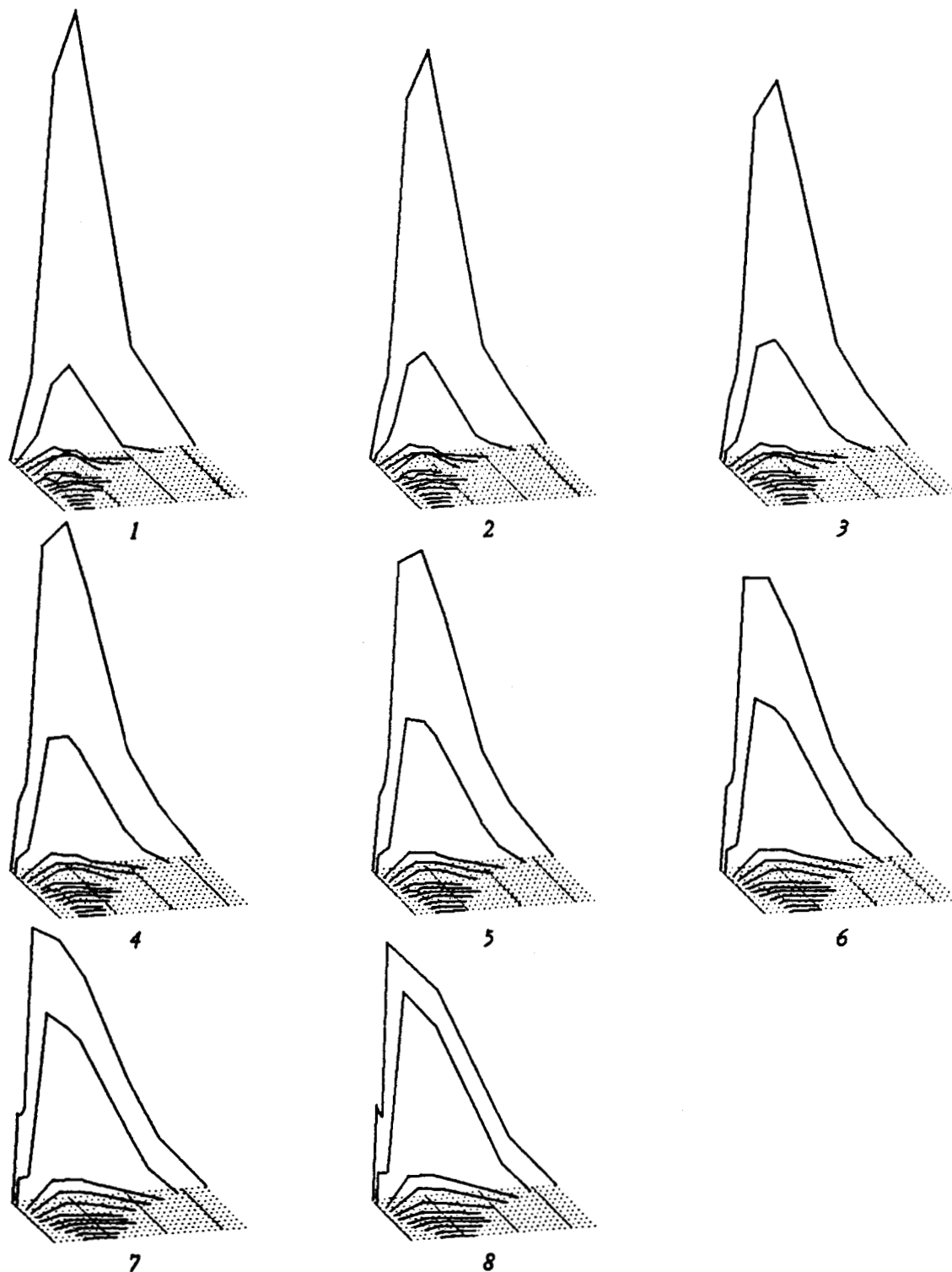


Fig. 18. Example of timbral transition in 14-percent steps. This transition starts from a piecewise-linear approximation to the functions from a cello tone and finishes at the piecewise-linear approximation to functions from a French horn tone.

pitch of the speech (if it is voiced), and the filter coefficients, all at regularly spaced points in time. This data can be used to synthesize the original utterance directly, or it can be modified. We can for instance, synthesize the utterance at different rates simply by updating the parameters (pitch, filter coefficients, voiced/unvoiced decision) faster or slower than they occurred in the original utterance. We could synthesize the

utterance at a different fundamental frequency, but with the original timing, simply by multiplying all the pitches by a fixed scale factor.

If one takes recorded speech and merely speeds up or slows down the waveform itself, the fundamental frequency of the speech is changed as well as the tone quality (Slawson [70]). By first analysing the speech, we can change the timing or the

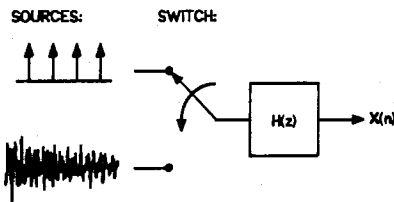


Fig. 19. A commonly used model of speech production. The model divides speech production into two parts: a source function, and a filter $H(z)$. The source function is either periodic or aperiodic. In the periodic case, a wide-band signal such as a pulse train is used. In the aperiodic case, white noise is used. The filter $H(z)$ is usually considered to be time-varying. For simplicity, it is usually taken to be an all-pole network of limited order (such as order 16).

fundamental frequency of the synthesized speech independently. Although it is not easy to keep the tone quality exactly constant while changing the fundamental frequency of the synthetic utterance, it is much more constant using this analysis-synthesis system than by, for instance, speeding up or slowing down a tape recording.

One can also resynthesize the signal using a complex waveform as excitation rather than a simple pulse train. This can produce a variety of most peculiar results! Petersen at the University of Utah ([55]–[57]) has experimented with using whole pieces of music as excitation sources. This produces the effect of the “talking orchestra,” where the character of the speech is also apparent. The intelligibility can be enhanced by choice of very wide-band sources, such as full orchestra and chorus (as opposed to, for instance, a string quartet).

The goal of this kind of modification is to give the composer a wide range of new “instruments,” both derived from natural sounds but distinct from natural sounds. The subtractive method provides an excellent opportunity for such combinations in that the sound generation process is already broken into two parts: the excitation and the spectral shaping. This method also provides easy control of time and pitch, in that once the analysis has been done, the synthesis can proceed at any rate, and the estimated fundamental frequency can be modified in any way desired.

Petersen has also experimented with synthesis of music instrument tones entirely by linear predictive coding. This produces a reasonable synthesis, but does not seem to produce the remarkable similarity to the original that the additive method does. It has been our own experience that none of the currently used subtractive methods produce synthetic speech or music of extremely high quality. When compared to the original, listening on headphones on a high quality audio system, the synthetic is always easily distinguishable from the original. Listeners complain that the synthetic is “buzzy.”

We should emphasize the fact that our standards for judging musical sound are much more critical than our standards for judging communications channels. We will tolerate telephone-quality speech for communication purposes, but we would be upset if our latest record purchase sounded like it had been transmitted through a voice-grade telephone channel. With this in mind, then, it is quite reasonable that the work on speech analysis and synthesis (“vocoder” applications) would not be directed to preserving very high fidelity. In communication work, it is often enough that the voice be highly intelligible and recognizable, though quality is still a consideration.

Our own experiments using extremely high order filters and more complex excitation functions seem to indicate that this

last increment of quality is not readily forthcoming. There seems to be a fundamental shortcoming in this method that prevents exact duplication of the original. The problem seems to be that although we can model the spectrum very accurately with high-order filters, we do not have good tools for modeling the excitation function.

We feel the advantage of this procedure is that it offers a way to model the spectrum of a signal as a time-varying digital filter. This filter can then be used to shape the spectra of other signals. If a method can be found for either modeling more precisely the excitation function or modifying the pitch and timing of the excitation function, then it might be possible to achieve very high fidelity using the linear prediction algorithm to compute the filter. If this were the case, then this technique would be somewhat more valuable than the additive synthesis technique because it essentially “decouples” the effects of pitch, time, and spectrum. In additive synthesis, the pitch and spectrum are intimately linked in that the amplitude function for a particular harmonic is applied to a different frequency when the harmonic changes frequency. Thus changing the pitch of an additive synthesis instrument scales the spectral shape also, whereas in the subtractive model, the pitch and the spectrum are independent.

Another advantage of the linear production method is that it is not highly sensitive to the excitation function. This means that it can compute a filter which matches, in a minimum squared error sense, the spectrum of an inharmonic waveform as well as that of a perfectly harmonic waveform. This is not true of the analysis for additive synthesis, which assumes a perfectly periodic waveform. The linear predictor is not sensitive to the fact that the spectrum it is matching is that of a harmonic nature or an inharmonic nature, except when the filter order is very high; then each harmonic is tracked separately in the periodic case. Although we do not know whether the experiment has been attempted to date, it would be interesting to attempt to simulate drum sounds this way. Many drums have totally inharmonic spectra which might be well simulated by modeling it as a time-varying filter with white noise as excitation.

SUMMATION FORMULA SYNTHESIS

This form of synthesis requires no analysis, a fact which is both an advantage and a disadvantage. The advantage is that no analog-to-digital conversion equipment is necessary, and thus this method is applicable to very small systems. The extreme simplicity of some of these formulas makes them quite attractive when compute power is limited. This method is readily applicable to what we have called “direct” synthesis. The disadvantage is that since no analysis is used, these tones can not be expected to exactly duplicate natural sounds. As it turns out, sometimes quite convincing replications can be produced, but there is no analytic technique that we know of which could have been used to design these replications.

The name “summation formula” comes from the way these formulae are derived. They almost all come from the summation of series, a topic that was extensively studied by Stirling in the 18th Century (Stirling [72]).

This class of synthesis techniques can be thought of as new musical “instruments” in that one must learn to “play” these instruments in a manner analogous to learning the violin. One must learn what settings of the controlling parameters produces what kinds of sounds. Since there is no analysis possible, the instrument must be treated in an empirical

manner. We explore its capabilities by experimenting with different settings of the controlling parameters, slowly building up a body of knowledge about how to obtain different kinds of sounds.

The parent of these methods is the frequency modulation equation which was first exploited for its timbral qualities by Chowning [13]. When stated as a synthesis technique, the output waveform is computed by the following formula:

$$X(n) = A(n) \sin \{2\pi f_c nT + I(n) \sin(2\pi f_m nT)\} \quad (9)$$

where $A(n)$ is the amplitude of the waveform. In most cases, this will be a simple slowly-varying function of time. It might have an *attack* portion, a *steady-state* portion and a *decay* portion much like waveforms from common music instruments. $I(n)$ is sometimes called the "modulation index." It is shown here as a function of time. Like the amplitude, it is usually simple and slowly varying. f_c is called the "carrier" frequency. f_m is called the "modulation" frequency. T is the time between samples. This says that the tone is completely determined by choosing the functions $A(n)$ and $I(n)$, and by choosing the other parameters f_c , f_m , and T . To understand how this makes music, let us examine the right side of equation (9) as a sum of sinusoids. The following trigonometric identity was taken from Abramowitz and Stegun [17]:

$$\sin(\theta + a \sin \beta) = J_0(a) \sin \theta + \sum_{k=1}^{\infty} J_k(a) \{ \sin(\theta + k\beta) + (-1)^k \sin(\theta - k\beta) \} \quad (10)$$

where $J_k(a)$ is the K th Bessel function at the point a , and $J_0(a)$ is the zeroth Bessel function at the point a .

We can relate this equation with (9) by making the identification $\theta = 2\pi f_c nT$, $\beta = 2\pi f_m nT$, and $a = I(n)$. The terms "carrier" frequency and "modulation" frequency are taken from FM broadcasting terminology (Crosby [18], Corrington [15]).

What this equation tells us is that we can produce a group of sinusoidal partials, the frequencies of which are separated by some frequency f_m . This means that we can produce a harmonic spectrum simply by relating f_c and f_m by a rational number. For instance, $f_c = f_m$ gives us a harmonic spectrum, with the amplitudes of the components determined by sums of pairs of Bessel functions of the modulation index a .

Using the fact that $\sin(-x) = -\sin(x)$ with (9) and (10), we can write out the Fourier series representation of the output waveform for the case when $f_c = f_m$:

$$\begin{aligned} \sin(\theta + a \sin \theta) = & \{J_0(a) - J_2(a)\} \sin(\theta) \\ & + \{J_1(a) + J_3(a)\} \sin(2\theta) \\ & + \{J_2(a) - J_4(a)\} \sin(3\theta) \\ & + \{J_3(a) + J_5(a)\} \sin(4\theta) + \dots \quad (11) \end{aligned}$$

This is just an expansion of (10) for the case of equal carrier and modulating frequencies. We can see that the identification of $\theta = 2\pi f_m nT$ makes it clear that (11) represents a harmonic series. The sinusoids are harmonics of ascending frequency. The amplitudes of these harmonics are sums of Bessel functions. Thus by controlling the modulation index a , we can control the spectral content of the output waveform. We must, however, gain some insight into how the Bessel functions behave as functions of the modulation index.

When the modulation index is zero, a pure sinusoid results because $J_0(0) = 1$ and all other Bessel functions are zero at the origin. This is clear from any of the above equations because when the index is zero, the equation is purely sinusoidal. As a gets larger and larger, more and more energy goes to the sidebands and the tone gets more and more "tinny" sounding. By controlling the index dynamically, we can simulate the time variant aspect of natural spectra. This is where the richness of the method arises.

Fig. 20 shows the variation in spectrum as the modulation index is swept from 0 to 8 (Chowning *et al.* [14]). Although there is quite a bit of activity in the amplitudes of the harmonics, we can see that the general trend is for the fundamental to decrease with increasing index as more and more harmonics become significant.

As an example of how we make musical tones with frequency modulation, we know that brass tones tend to start out as pure sinusoids and quickly evolve to a more complex spectrum (Risset and Mathews [60], Benade [9], [10]). We can easily simulate this behavior by increasing a very quickly during the attack portion of the tone. In fact, this produces a quite realistic brass-line tone. No one would mistake it for the sound from a professional trumpet player, but it is useful, especially for its simplicity, both computational and conceptual. Fig. 21 shows a perspective plot of the spectrum of a brass-like FM tone. The small upper plots show the functions that were used for $A(n)$, the amplitude, and $I(n)$, the index. Possibly the most realistic trumpet simulation using frequency modulation was done by Morrill [46], [47] at Colgate University. To increase the realism, many subtle refinements were added, such as varying the frequency of the tone slightly during the attack portion.

Other relations between f_c and f_m give other interesting results. For instance, $f_m = 2f_c$ gives odd harmonics only and is useful for clarinet-like sounds. We can see how this works by expanding (10) for the case where $\beta = 2\theta$:

$$\begin{aligned} \sin(\theta + a \sin(2\theta)) = & \{J_0(a) + J_1(a)\} \sin(\theta) \\ & + \{J_1(a) - J_2(a)\} \sin(3\theta) \\ & + \{J_2(a) + J_3(a)\} \sin(5\theta) \\ & + \{J_3(a) - J_4(a)\} \sin(7\theta) + \dots \quad (12) \end{aligned}$$

We can see that no even numbered harmonics will be present in the output waveform. Since the clarinet is a cylindrical instrument that is closed at one end, its natural modes of vibration will favor the odd harmonics (Benade [8], [10]). By simulating that aspect of the spectrum of the instrument with frequency modulation, a sound that is clarinet-like can easily be produced.

We see from these examples that the carrier frequency, f_c , sets the *origin* of the spectrum and the modulation frequency f_m determines the *spacing* of the partials.

Setting $f_m = \sqrt{2} f_c$ gives an inharmonic complex that can be used to make bell, gong, drum, and other non-periodic tones. Many other examples are given in Chowning [13], complete with specific parameter values for specific sounds (a useful "cookbook" of FM instruments).

If we wish to implement this algorithm on a system with limited facilities, such as a microprocessor system, we must look carefully at the arithmetic operations involved in the computation described by (9). We can see that there are two sinusoids, two multiples, and one add. We can perform the

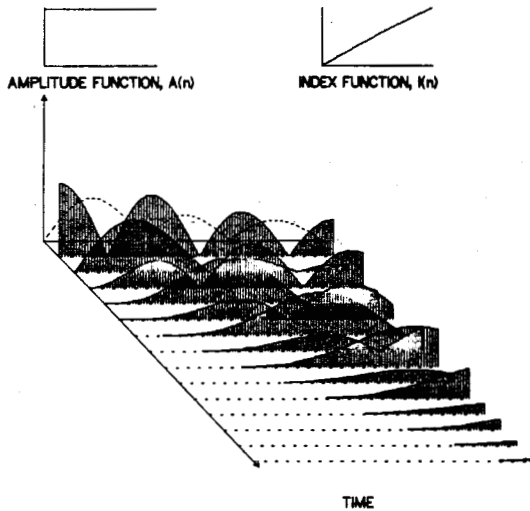


Fig. 20. Perspective plot of the spectrum of a frequency modulation signal with $f_m = f_c$ as the modulation index is swept from 0 to 8. The small plots above the perspective plot show the amplitude (on the left) and the modulation index (on the right). For this plot, the amplitude was held constant. The axis coming out of the page is frequency, starting at zero at the origin. The dotted line at frequency = 0 is a dc term that always accompanies the FM signal.

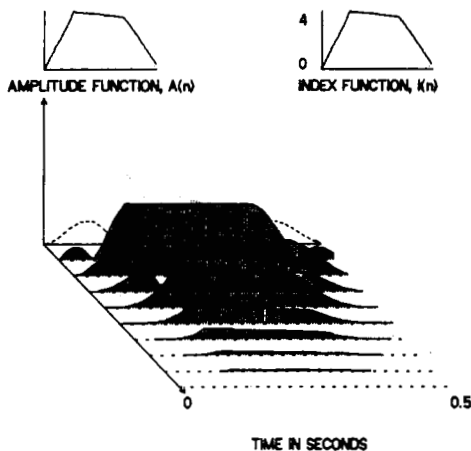


Fig. 21. Perspective plot of the spectrum of a frequency modulation signal of a brass-like tone. In this case, f_c was set equal to f_m . The index function is the same shape as the amplitude function. The index starts at 0 and goes to a maximum of 4, then decays back to zero. The entire tone takes about 0.5 s. We can see how the fundamental comes in first, followed by higher and higher harmonics. This mimics the general spectral trend of the trumpet [9], [60]. These controlling parameters are also applicable to other summation formulae. For instance, either equation (15) or equation (16) could be used to produce a brass-like tone with this same amplitude function and same index function. Each of the tones will be different, but all will have a brass-like character. Likewise, any of the equations (19) through (22) could be used with the number of harmonics (N) set to about 8 (in the finite cases, equations (19) and (21)) and the maximum index scaled to be about 0.82 (rather than 4 as above).

sinusoids by table (or ROM) lookup. Addition is also simple enough. The multiplies often present a problem for small systems. As described by Saunders [63], there are at least two ways for eliminating the multiplications in FM. The first is to examine the following trigonometric identity:

$$\sin(s + d) + \sin(s - d) = 2 \cos(d) \sin(s). \quad (13)$$

If we make the identification $a = 2 \cos(d)$, we can see that we can simulate scaling a sinusoid by a factor from -2 to +2

by computing sinusoids of unit amplitude and adding them together. All this means is that at some point we must convert our controlling functions ($A(n)$ and $I(n)$) into displacements (d above) by taking the arccosine of half the functions. This can be done beforehand for a variety of functions and stored for later real-time use. One problem with this method is that of accuracy. If the table used to store the unit-amplitude sinusoids is not of sufficient length, we find the amplitudes we can realize limited to discrete steps that are determined entirely by the length of the sine table. Insufficient table length can produce "roughness" in the sound.

We need not modulate the sinusoid with another sinusoid. In fact, modulating with a triangle wave (Saunders [63], Holway and Mullen [28]) eliminates a multiplication from the algorithm and makes it possible to implement the algorithm on a microcomputer with limited arithmetic with several voices in real time. This seems to be one of the most promising methods for achieving real-time operation with limited facilities.

To explain this more precisely, let us examine the behavior of the following synthesis technique:

$$X(n) = A(n) \sin \{2\pi f_c nT + I(n) R(2\pi f_m nT)\} \quad (14)$$

where $R(x)$ is a triangle waveform of amplitude -1 to +1. This is the same as (9) except that we have substituted a triangular periodic wave (denoted $R(x)$) for the modulating sinusoid. Holway and Mullen [28] have computed a simple closed-form expression similar to (10) for the case of trapezoidal modulating waveforms. Our triangle wave is just a special case of the more general trapezoidal function. Their expression confirms our own experience that this sounds very much like the purely sinusoidal modulating function and very similar tones can be made this way.

The reason for going to a triangular modulating wave is that the combined function, $I(n) R(2\pi f_m nT)$, can be synthesized almost entirely without multiplications and places no unusual constraints on the table size required for the sinusoid. We can see this by considering how we write a program to generate this waveform. We can produce the next point of the scaled triangle by just adding or subtracting the value $2I(n) f_m T$ from the previous point. If the resulting value exceeds the range $-I(n)$ to $+I(n)$, the value must be brought into range. The following is an Algol-like program that computes the next point of the scaled triangle wave $I(n) R(2\pi f_m nT)$:

```

if  $I(n) \neq I(n-1)$  then  $inc \leftarrow 2 * I(n) * f_m T$ ;
if ascending then
begin
  value  $\leftarrow$  value + inc;
  if value  $> I(n)$  then
  begin
    value  $\leftarrow 2 * I(n) -$  value;
    ascending  $\leftarrow$  false;
  end;
end
else begin
  value  $\leftarrow$  value - inc;
  if value  $< -I(n)$  then
  begin
    value  $\leftarrow -2 * I(n) +$  value;
    ascending  $\leftarrow$  true;
  end;
end.

```


This program is meant as an example to show how the scaled triangle can be computed with few multiplies. There must be some initialization of the variables *value*, *inc*, and *ascending*, together with declarations and some definition of $I(n)$. Presumably if $I(n)$ were a piecewise-linear function, the increment (variable *inc* in the above program) could be updated by additions also. A multiplication would only be required at the breakpoints of the function.

As examples of the attractiveness of FM even without the above simplifications we might mention in passing two (of several) sites that have real-time FM capability:

At Dartmouth, under the direction of Appleton [2], a digital synthesizer capable of realizing up to 4 voices of FM in real time has been built and is currently being used both for teaching composition and for advanced composition. This device can be used as one 4-voice synthesizer, or can be used by four different people simultaneously as 1-voice synthesizers.

At MIT, under the direction of Vercoe [74], there is a compositional facility using a general purpose computer (PDP-11/45) which just through software can achieve 4 voices of FM in real time. The MIT facility has an advanced graphics terminal, a large disk, an organ keyboard, high-quality digital-to-analog converters, tablet input, and results in a quite powerful and flexible compositional facility.

BUT WHERE DO THE NUMBERS COME FROM?

Making up FM instruments takes quite a bit of intuition about what changes in the parameters make what changes in the sounds. This intuition is of course necessary when any changes are made in the intermediate analysis data of any of the analysis-based methods. In our introduction to computer music, in the Stanford University Music Department, we attempt to give the students a balanced exposure to programming, signal processing for the musician, and psychoacoustics, as well as quite a bit of "hands-on" experience with the sound system. The point is that introducing the computer as a new "instrument" of the orchestra requires that we teach specifically for this instrument if we expect it to be used with skill and imagination.

It is interesting to note that these formulae have been known for quite a while; in some cases, for dozens of years. It is only recently that they have been applied to produce musical sounds. Engineering students have been learning the FM equation for years, but few people ever thought of listening to the modulated waveform itself. Although frequency modulation has been used by musicians with analog synthesizers, the precision of voltage-controlled oscillators is not sufficient to produce harmonic tones such that f_c and f_m are related by a rational number. We also require cases where the frequency goes negative which occurs when the modulation index is sufficiently high. With digital processing, these features are quite easy to provide. Both the precision and versatility are readily available. The only barrier right now is that it is difficult to produce real-time operation cheaply using digital processing. This presumably will be no barrier in the future.

As an example of some of the kinds of tones that can be made using frequency modulation synthesis, Table I presents the controlling parameters for three different tones: a brass-like tone, a clarinet-like tone, and a bass drum-like tone. In the table, the carrier and modulation frequencies are expressed as multiples of the frequency of the desired sound. With the bass drum, this frequency is not well defined, since

TABLE I

BRASS-LIKE TONE:
$f_c = f_0$
$A(n) = (0, 0.5 \times 100, 1 \times 300, 1 \times 400, 0)$
$K(n) = (0, 1 \times 50, 4.5 \times 400, 4.5)$
CLARINET-LIKE TONE:
$f_c = 3f_0$
$f_m = 2f_0$
$A(n) = (0, 0.5 \times 100, 1 \times 300, 1 \times 400, 0)$
$K(n) = (0, 2 \times 100, 1 \times 400, 1)$
BASS DRUM-LIKE TONE:
$f_c = f_0$
$f_m = 1.414 f_0$
$A(n) = (0, 0.75 \times 50, 1 \times 100, 0.5 \times 300, 0.2 \times 1000, 0)$
$K(n) = (0, 1.5 \times 100, 1 \times 300, 0.7 \times 1000, 0)$

Controlling parameters for three simple FM instruments. In these instruments, f_0 is the desired pitch of the resulting tone. For the brass-like tone, the carrier and modulation frequencies are the same as the desired frequency, but in the other two tones, this is not the case. The brass-like and clarinet-like tones were generated originally using a frequency of $f_0 = 300$ Hz. The bass drum-like tone used a very low frequency of $f_0 = 40$ Hz. The amplitude and index functions are given in piecewise-linear form as ordered pairs. Each pair represents one breakpoint of the function. The first number of each pair is the time in milliseconds of that breakpoint and the second number is the value at that time. These points should be connected by straight lines. In the bass drum-like tone, the amplitude starts at a non-zero value. This indicates an instantaneous attack. The drum tone is just "turned on" at time zero. This produces that characteristic sharp percussive sound.

the bass drum is largely inharmonic. In this case, the frequency refers more to the rough spectral location of the partials, rather than corresponding to an actual pitch. We generated these examples by intuition and trial-and-error. The frequencies used for the brass-like tone and the clarinet-like tone were 330 Hz. The frequency of the brass drum-like tone was 40 Hz. The amplitude and index functions are given as piecewise-linear approximations. The breakpoints of these functions are given as ordered pairs. The first number of each pair is the time in milliseconds of the breakpoint and the second number is the value at that point. These points should be connected by straight lines. Note that the drum-like tone does not start at zero. It has an instantaneous attack that produces a crisp percussive sound.

These examples are very simple and are intended only as guidelines for getting started with frequency modulation synthesis. Much more interesting sounds can be made by more complex usage of the formulas. For instance, with frequency modulation, we might choose more than one modulating waveform, or perhaps different waveforms than sinusoids.

AND ARE THERE OTHERS?

There is no reason to expect that FM is the only such formula. In fact, a quick search of the tables of sums yields several new and interesting such formulas. Probably the most complete such table is that of Hansen [26], although far more comprehensive than we need for musical purposes.

The most direct descendants of the FM equation are the following two formulas:

$$e^{a \cos \beta} \sin(\theta + a \sin \beta) = \sum_{k=0}^{\infty} \frac{a^k}{k!} \sin(\theta + k\beta) \quad (15)$$

$$e^{a \cos \beta} \sin \theta = I_0(a) \sin \theta$$

$$+ \sum_{k=1}^{\infty} I_k(a) \{ \sin(\theta + k\beta) + \sin(\theta - k\beta) \} \quad (16)$$

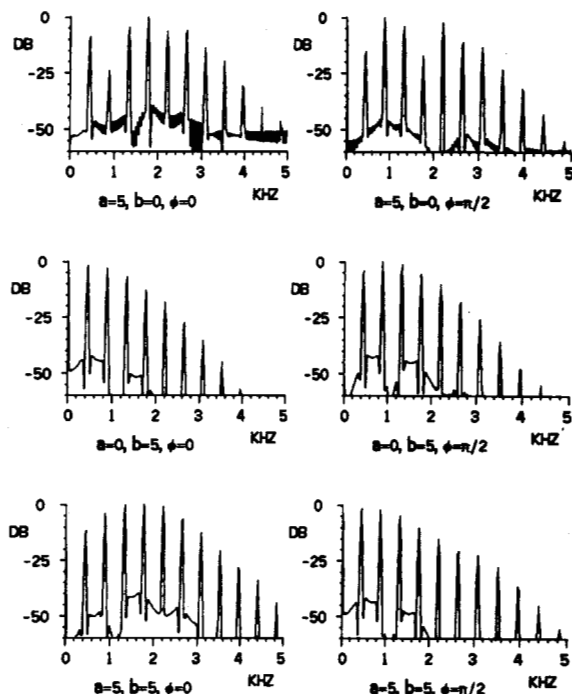


Fig. 22. Spectra of waveforms produced by equation (17). In this case $f_c = f_m = f_g = 440$ Hz. With $b = 0$, the method is pure frequency modulation as shown in equation (9). When $a = 0$, the method is as shown in equation (16). When $a = b$, it is equivalent to equation (15). Since the spectrum changes with phase angle, we have included both $\phi = 0$ and $\phi = \pi/2$ here. We can see that the spectra for $b = 0$ (pure FM) is the most irregular and for $a = 0$ is the most regular. We can see clearly the cancellation of the lower harmonics due to reflection of negative frequencies to positive. By adjusting ϕ , the phase angle, we can cause the reflected sidebands to cancel or reinforce.

where $I_k(a)$ is the K th modified Bessel function at the point a , (note: this should not be confused with $I(n)$, the modulation index as a function of sample number, n) and $I_0(a)$ is the zeroth modified Bessel function at the point a . Both of these (like all of the following formulas) are more computationally complex than the simple FM equation, but all retain the conceptual simplicity. We have shown only the trigonometric summation forms. To make these into synthesis techniques, we need only take the left sides of these equations, substitute $2\pi f_c nT$ for θ , $2\pi f_m nT$ for β , and $I(n)$ for a . We should also multiply the expression by $A(n)$ to allow for dynamically changing amplitude. This gives us expressions for the n th point of the output waveform as functions of the controlling parameters f_c , f_m , $I(n)$, and $A(n)$.

Each synthesis technique thus produced gives a waveform that has its own characteristic "sound." We can, for instance, mimic a brass-like tone with any of these formulae and we get a different sound from each one. One difference is that the original FM formula exhibits partials that are "sparse." That is, some partials may be of insignificant amplitude, whereas all the other forms exhibit partials that have amplitudes that change smoothly from one partial to the next. In fact, equation (16) exhibits partials that are monotonic from the center frequency.

Notice that we can generalize (15) to the following:

$$e^{b \cos \gamma} \sin(\theta + a \sin \beta). \quad (17)$$

There is no right-hand side to this equation because the trigonometric summation for this expression is quite complicated, but can easily be derived from (10) and (16). We have

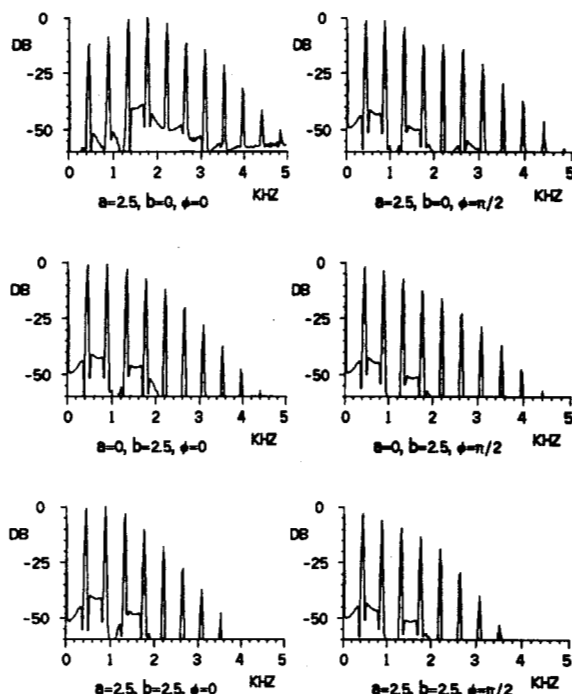


Fig. 23. Spectra of waveforms produced by equation (17). In this case, $f_c = f_m = f_g = 440$ Hz. With $b = 0$, the method is pure frequency modulation as shown in equation (9). When $a = 0$, the method is as shown in equation (16). When $a = b$, it is equivalent to equation (15). Since the spectrum changes with phase angle, we have included both $\phi = 0$ and $\phi = \pi/2$ here. We can see that the spectra for $b = 0$ (pure FM) is the most irregular and for $a = 0$ is the most regular. We can see clearly the cancellation of the lower harmonics due to reflection of negative frequencies to positive. By adjusting ϕ , the phase angle, we can cause the reflected sidebands to cancel or reinforce.

given only the left-hand side, which is all that is needed to make a synthesis technique. By setting a and b to various values, we can get both equation (10) ($b = 0$), equation (15) ($a = b$), and equation (16) ($a = 0$). The spectra will change smoothly when we change from one form to the other. For instance, the tone can start with a "full" spectrum, like that produced by (16), and change to a "sparse" spectrum, like that produced by (10). By setting the relation among β , θ , and γ , we can get both harmonic and inharmonic spectra, as well as mixtures of the two.

Fig. 22 shows spectra of signals produced by using (17) as a synthesis technique with $\theta = 2\pi f_c nT$, $\beta = 2\pi f_m nT$, and $\gamma = 2\pi f_g nT$. In the figure, all the frequencies are set to 440 Hz ($f_m = f_c = f_g = 440$ Hz). There is, however, an additional complication. A phase angle can be added in the carrier sinusoid that affects the way reflected sidebands are combined. We can demonstrate this by rewriting (11) including a constant phase angle ϕ in the carrier sinusoid:

$$\begin{aligned} \sin(\theta + a \sin \theta + \phi) &= J_0(a) \sin(\theta + \phi) - J_2(a) \sin(\theta - \phi) \\ &\quad + J_1(a) \sin(2\theta + \phi) + J_3(a) \sin(2\theta - \phi) \\ &\quad + J_2(a) \sin(3\theta + \phi) - J_4(a) \sin(3\theta - \phi) \\ &\quad + J_3(a) \sin(4\theta + \phi) + J_5(a) \sin(4\theta - \phi) \\ &\quad + \dots \end{aligned} \quad (18)$$

This equation shows that by setting ϕ to values from 0 to $\pi/2$, we can get the reflected sidebands to cancel or to reinforce. This effect is clearly shown in Fig. 22. The amplitudes of the lower harmonics can be changed significantly by varying this phase angle. Thus the phase angle gives us yet another degree

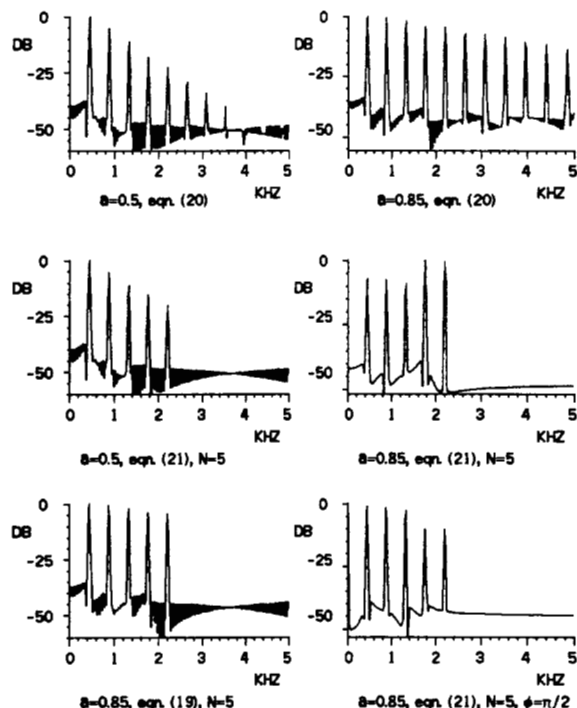


Fig. 24. Spectra of signals produced by synthesis techniques based on equations (19), (20), and (21). In these cases $f_c = f_m = 440$ Hz. The upper plots show the infinite sum case for two different values of a , equation (20). The center plots show the finite sum case for two different values of a , equation (21). Equation (19) was used for the lower left plot with the result that there are no reflected sidebands to interfere with the lower harmonics. In the lower right plot, the phase angle ϕ has been set to $\pi/2$ to show how the reflected sidebands can be used to cancel or reinforce the lower partials.

of freedom when dealing with summation formulas. Please note that here we have omitted the term at zero frequency.

Fig. 23 shows settings of the values of a and b that do not reduce to simple instances of (15) or (16), except when $a = b$. All the above spectra are infinite trigonometric series. We can also introduce a class of finite series, with their infinite limits:

$$\sum_{k=0}^N a^k \sin(\theta + k\beta) = \frac{\sin(\theta) - a \sin(\theta - \beta) - a^{N+1} [\sin\{\theta + (N+1)\beta\} - a \sin(\theta + N\beta)]}{1 + a^2 - 2a \cos(\beta)} \quad (19)$$

$$\sum_{k=0}^{\infty} a^k \sin(\theta + k\beta) = \frac{\sin(\theta) - a \sin(\theta - \beta)}{1 + a^2 - 2a \cos(\beta)} \quad (20)$$

$$\sin(\theta) + \sum_{k=1}^N a^k \{\sin(\theta + k\beta) + \sin(\theta - k\beta)\} = \frac{\sin(\theta) (1 - a^2 - 2a^{N+1} [\cos\{(N+1)\beta\} - a \cos(N\beta)])}{1 + a^2 - 2a \cos(\beta)} \quad (21)$$

$$\sin(\theta) + \sum_{k=1}^{\infty} a^k \{\sin(\theta + k\beta) + \sin(\theta - k\beta)\} = \frac{(1 - a^2) \sin(\theta)}{1 + a^2 - 2a \cos(\beta)} \quad (22)$$

Equations (20) and (22) are just the limits of (19) and (21) as $N \rightarrow \infty$. In these two infinite series, a must be strictly less than 1. Equation (21) was derived by taking two copies of (19), one with positive β and one with negative β , scaling one by a and adding. This gives a two-sided spectrum. The form in equation (22) is the simplest form we know outside of FM itself.

Fig. 24 shows some typical spectra for these formulas. In the upper two plots, we contrast the spectra for two values of a , 0.5 and 0.85, using the infinite summation shown in (20). The second pair of plots contrasts the spectra of (21) for the same two values of a . The lower left plot shows the finite

one-sided form of (19). The lower right plot shows the spectrum from (21) again, but the phase angle ϕ has been set to $\pi/2$, which causes the reflected sidebands to reinforce rather than cancel.

The only problem with these particular formulas is that they must be normalized. FM is automatically normalized, for its amplitude is always between +1 and -1. The FM derivatives such as are shown in (15)–(17) can be normalized to that scale by dividing by e^a . Equations (19) through (22), however, can have amplitudes that vary over quite wide ranges as a is varied. In Moorer [50], a series of power normalization functions are given which can be used to scale these equations. These normalization functions are quite complex. They are, however, functions of only a few parameters. They could easily be computed beforehand and stored in tables. This would mean that the normalization function would not have to be computed on each sample, but only looked up in a precomputed table.

One way that new closed forms of summations are found is through the use of mathematical techniques of accelerating or rearranging series so as to improve the convergence properties. Gosper [23], [24] reported some new results in this field which may be of some use in finding new families of closed forms of trigonometric summations.

SPATIAL RELATIONS

The environment in which music is played has always been a great concern. Recording studios are capable of imposing that "concert hall" sound on musical sound even without the aid of a concert hall. With the advent of stereo, and now quadraphony, new uses of simulated environment are possible (Chowning [12]).

There are two aspects of the spatial environment that we can simulate: the perceived location of a sound source and the room reverberation. There are commercially available devices to simulate both of these aspects through analog electronic equipment. Fig. 25 shows a quadraphonic layout that can be

used to simulate location. Four speakers are located at corners of a square. The listener or listeners are seated at the center of the square. To produce the illusion that a sound is coming from a particular place in space, we need only distribute the energy to the two speakers nearest that place. There are commercially available "joysticks" that can accomplish this distribution easily. There are also commercially available reverberators for producing that "concert hall" quality.

There are many advantages to using digital processing instead of analog equipment for spatial illusions. The first reason is flexibility. When you purchase a standard analog commercial

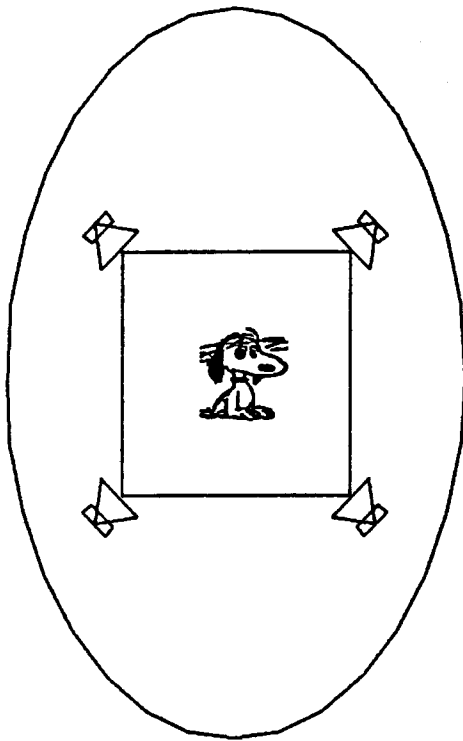


Fig. 25. Plot of quadrasonic layout. The listener is positioned in the center of four speakers which are arranged in a square (ideally). The ellipse shown is a hypothetical trajectory that we might want the sound location illusion to follow. The ellipse has been approximated here with 32 line segments and thus is not perfectly smooth.

reverberator, it has a characteristic "sound" that is strictly related to its construction. With digital processing, we can "reprogram" the reverberator for a variety of different sounds. Another reason is control: we have the option to control more aspects and different aspects of the spacial illusion. For instance, it is quite simple to add Doppler shift to a sound simply by linking its pitch to its radial velocity with respect to the listener. We can set up elaborate "trajectories" for sounds to follow which can be carried out very precisely with many simultaneous sounds following independent trajectories.

The heart of spatial positioning is reverberation and distribution of energy among a multiplicity of speakers. We are using four channels at Stanford (discrete quadrasonic), but plan to explore the utility of more channels in the future. It is clear that angular placement of the illusory source can be achieved by distributing the energy to the two speakers around the desired position, but for the illusion of distance, we must appeal to perceptual research for the appropriate cues. It so happens that some of the most relevant work on this was done by Von Békésy [6], who discovered that the distance cue is the *ratio* of the reverberant sound to the direct signal. It comes as a surprise to many people that the amplitude of the signal is not a direct distance cue. Experiments in anechoic chambers (Gardner [21]) demonstrate this in most dramatic ways. Thus, the distance cue reduces to the problem of producing convincing artificial reverberation.

Although we could just record the impulse response of a room (a research project in itself), the convolution implied by a 2- to 4-s reverberation time is clearly outrageous. Schroeder [66]–[68] proposed a way to produce the illusion of a large concert hall without having to duplicate the enormous convo-

lution that the hall does for us. Using this simplified form of artificial reverberation does not allow for the exact duplication of a specific concert hall, but analogous to the summation formulas, gives us reasonable sounding reverberation without undue effort.

The basis of the Schroeder method is the concept of the "unit reverberator." Schroeder discussed two such units, to which we have added two more. These are shown along with their pole-zero plots in Figs. 26 and 27. (Note: we have put Schroeder's all-pass network in canonical form, thus eliminating one unnecessary multiply.) To design a reverberator, we choose a combination of these unit reverberators, and then set their delays and gains to produce the desired effect. These gains and delays (G and m in the figures) are usually set at the beginning of a piece and left constant throughout the piece. Again, we know of no analytic way to choose these things, but have developed a series of guidelines for the composer. These guidelines have been embedded in a program for setting up reverberators.

Although we call the building blocks "unit reverberators," a single such unit by itself does not produce high quality reverberation. The object is usually to get "smooth" sounding reverberation, free of repetitive echos. The difference between "echo" and reverberation is that with pure echo, there are clear repetitions of the signal being echoed, but with reverberation, there are no well-defined repetitions, but instead a "diffuse" sound results. Both reverberation and echo have musical uses. Smooth reverberation seems to be somewhat more difficult to achieve, however.

Recently, we have been favoring the allpass networks because we wish to separate the reverberation from the coloration of the sound. This gives us independent control of each aspect of the environment. We can "soften" the room, for instance, by low-pass filtering the reverberant signal as separate from the direct signal, leaving the direct signal unchanged.

To design a reverberator, we usually have some idea of the kind of room we wish to simulate. A large room would have a very long reverberation time, whereas a small room might have a very short reverberation time. Starting with the design ideal, we must then choose the number of units, the type of the units, and the gains and delays (G and m) of each unit. Fig. 28 shows impulse responses of 3-unit reverberators. We have used here only 3 unit reverberators to generate each of the plots in Fig. 28. Normally, we would use 5 to 7 units for an actual musical application. The first plot (upper left) shows the impulse response of a relatively smooth sounding reverberator. It is the cascade of 3 all-pass units, such as are shown in the upper part of Fig. 27. In each of the six cases shown in Fig. 28, the lengths of the delays are given in milliseconds. These numbers would have to be converted to an integral number of samples for practical use. We choose the prime number closest to each of the integer delays to assure that the delays lengths, in samples, are irreducible. This produces the most "dense" impulse response. The first plot in Fig. 29 shows what happens if these delays are reducible. It is clear that the density is reduced in this case. In Fig. 28, only the first plot is a desirable impulse response. The others exhibit regular strong peaks that are heard as discrete echos. All of the figures were produced by starting with the gain and delay of the first unit and generating the gains and delays of succeeding units by multiplying by constant factors. For instance, in the first plot, the gains were multiplied by 0.95 at each stage and the delays were multiplied by 0.8 at each stage.

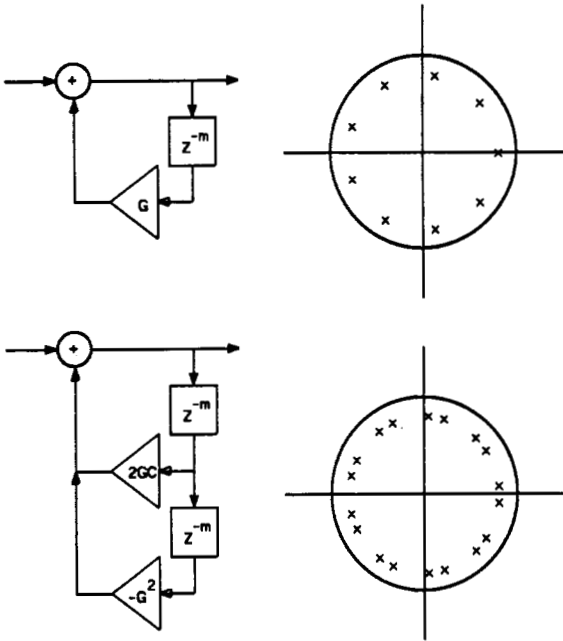


Fig. 26. Flow diagram and pole-plot for two unit reverberators. The upper figure is the standard all-pole comb filter. The root locus is shown for $m = 9$. The lower figure is an oscillatory filter. The impulse response of this unit will be a decaying sinusoid of frequency θ/m . In both filters, G controls the decay. The roots are all on a circle the radius of which is $G^{1/m}$. The term C stands for $\cos(\theta)$. In an actual musical context, m would be chosen to be a prime integer from about 50 to 2000.

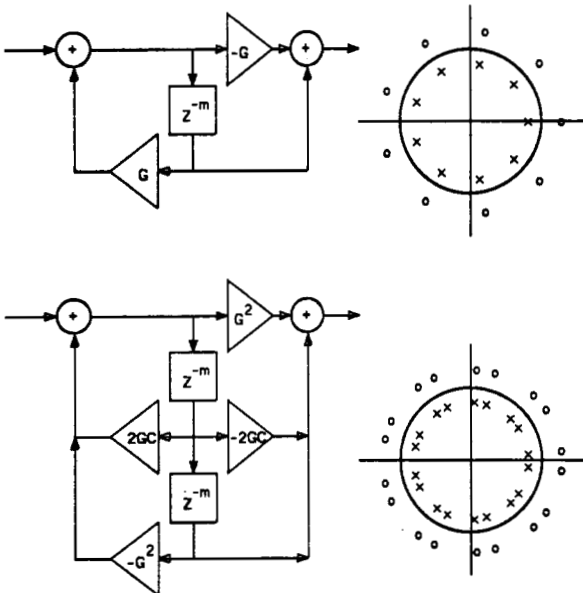


Fig. 27. Flow diagram and pole-zero plot for two all-pass unit reverberators. The upper figure is the standard simple all-pass. It is equivalent to the form given by Schroeder. The root locus is shown for $m = 9$. The lower figure is an oscillatory filter. It is also an all-pass. The impulse response of this unit will be a decaying sinusoid of frequency θ/m . In both filters, G controls the decay. The poles are all on a circle the radius of which is $G^{1/m}$. The zeros lie on a circle of radius $G^{-1/m}$. The term C stands for $\cos(\theta)$. In an actual musical context, m would be chosen to be a prime integer from about 50 to about 2000.

In the figure, the gain multiplier is represented by GM and the delay multiplier by DM. We can see that the smoothest, most diffuse impulse responses occur when the gain multiplier and

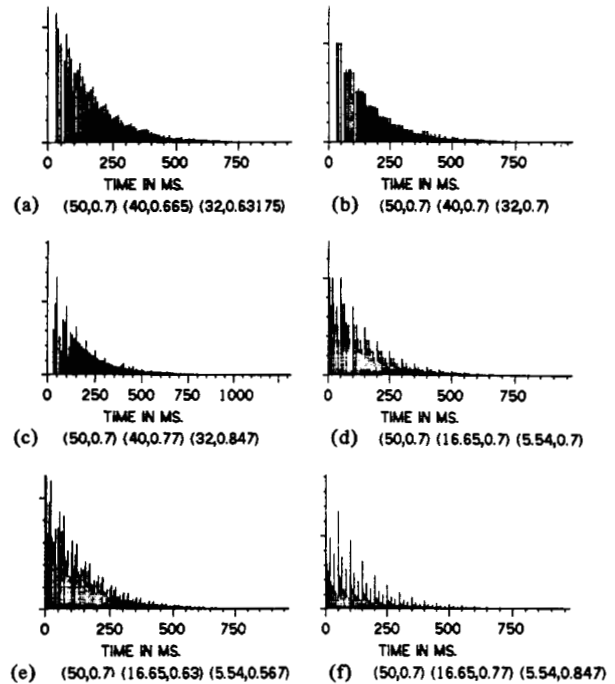


Fig. 28. Impulse responses of different reverberators. Each reverberator consists of three all-pass unit reverberators in series. The delays and gains of each unit are shown in parentheses below each plot. The first number is the delay in milliseconds and the second number is the gain. When the delay is converted to an integer number of samples, the nearest prime number should be chosen so as to assure irreducibility of the delays. In each of these cases, there is a relation among the delays and gains of the units for a particular plot. Each of the three units is derived from the previous one by multiplying the delay by a factor and the gain by a factor. The first unit in each case has a 50 ms delay and a gain of 0.7. (a) DM (delay multiplier) = 0.8 and GM (gain multiplier) = 0.95. (b) DM = 0.8, GM = 1. (c) DM = 0.8, GM = 1.1. (d) DM = 0.333, GM = 1. (e) DM = 0.333, GM = 0.9. (f) DM = 0.333, GM = 1.1. Of these six, only (a) is acceptably smooth. All the others show some degree of objectionable echo; (f) seems to be the worst of these, as we can see by the very high regular spikes.

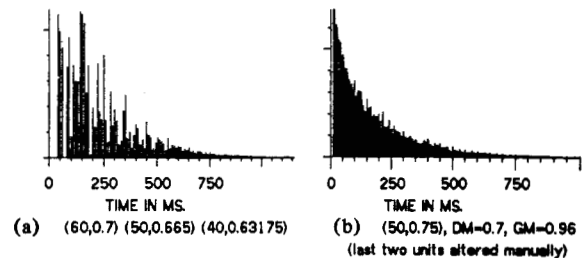


Fig. 29. Impulse responses of different reverberators. (a) Shows what happens when the delays in samples are not prime numbers. In this case, the delays were 60 ms, 50 ms, and 40 ms. We can see that the impulse response is not as dense as other impulse responses, and that pulses seem to be equally spaced. This causes objectionable echo in the sound. (b) Shows the impulse response of a 5-unit all-pass reverberator. The first unit has a delay of 50 ms and a gain of 0.75. Each successive unit has the gain set to 0.96 of the previous unit's gain and the delay set to 0.7 of the previous unit's delay, except that the last two units were altered manually from this regular geometric progression to give a smoother response. The delays of the last two are longer and the gains are slightly lower than the regular values. The delays were set to the prime number of samples nearest to Table II.

the delay multiplier are carefully chosen to be somewhat similar (but not equal). When they are very different, periodic spikes tend to appear. The worst case seems to be when one of the multipliers is less than one and the other multiplier is greater than one.

In general, however, one cannot rely on this simple rule for more than about three unit generators. For longer reverberators, the last units must be manually adjusted for smooth response. The impulse response of such a reverberator is shown in Fig. 29(b). Here, the first three units were chosen by multiplying the gain by 0.96 and the delay by 0.7, but the gains and delays of the last two units were adjusted by hand using an interactive program which displayed the impulse response after each change. By eliminating disproportionate spikes from the impulse response, we can assure ourselves of a smooth response.

There is another problem with distributing reverberation to multiple channels. If the same reverberation is distributed to all channels, a very artificial sound results. This is probably because natural reverberation has a spatial dependency, so that different directions present different reverberation. We can simulate the effect by using a different combination of delays and gains for each channel, but this is quite inefficient. A somewhat simpler scheme is to allow "branching" in the reverberator as shown in Fig. 30. By keeping the path common up until the lower levels of the reverberator, a great savings in computation is realized. The final units assure that the reverberant signal will be scrambled enough so that a rich, omnidirectional sound is perceived. Note that the signals from the different channels are not "uncorrelated" in the statistical sense with this branching reverberator. Note also that the reverberant signal delivered to each channel is still flat: each path of the reverberator is all-pass. Of course as soon as the sound gets out into the room, it can not be said to be flat any more, but by preserving the all-pass nature of the reverberator, we assure that at least the processing which we can control adds no coloration of its own. Fig. 31 illustrates this with a schematic representation of a rectangular room with a single point source and a point listener. Shown in the figure are four single-reflection paths and one direct path. It is clear that the time delay (length) of each of these paths are different. This being the case, there is no way that the frequency response between the source and the listener can be made perfectly flat, except in an anechoic chamber.

We are now in a position to give actual parameters for two example quad reverberators. Table II gives the delays and gains. The first set of parameters results in a reverberator that has a total reverberation time of about 1.2 s. This time is defined as the time it takes for the impulse response to decay down 60 dB. The second reverberator has a total reverberation time of approximately 1.5 s. Each reverberator is 5 stages and uses "branching" as shown in Fig. 31. The first three unit reverberators are common to all four channels. The fourth stage branches into two units, then each of those units branch into two more units. The total reverberation time is equalized on every path from the first unit to the last, so that the reverberation sounds very similar in each of the four channels. Each reverberator starts with factors of 0.7 between the delays of adjacent stages and a factor of 0.96 between the gains of adjacent stages. Again, the gains and delays for the last two stages were adjusted manually with an interactive display program. The shortest delays are placed last so as to minimize the total amount of memory required to store the delay arrays in the computer.

Unfortunately, doing spatial positioning in the above described manner does not recognize one of the most important positional cues: the time delay. People seem to be extremely

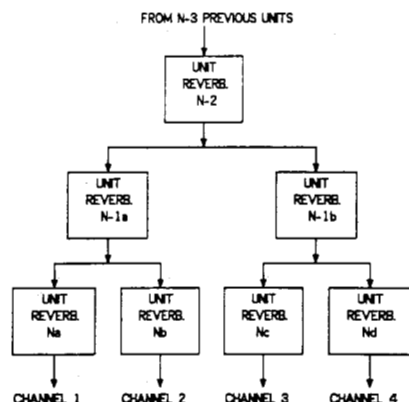


Fig. 30. Last two stages of an N -stage reverberator designed for use with 4-channel sound. Here the signal "branches" into separate and independent unit reverberators at both the $(N-1)$ th level, labeled $N-1a$ and $N-1b$, and the N th level, labeled Na , Nb , Nc , and Nd . These would presumably be very similar but would still use incommensurate delays. This assures that there is no audible "correlation" between the channels. Care must be taken to assure that the total reverberation time is very similar on each path. If all-pass character is not necessary, then simpler networks may be used.

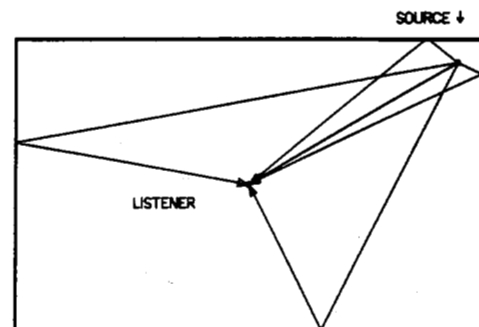


Fig. 31. Rectangular room with a sound source in the corner and a listener in the center. The direct sound path from the source to the listener, as well as four single-reflection paths are shown. Since each path has a different transit time for sound, the overall transfer function from source to listener cannot be made flat, except by eliminating the reflections (anechoic chamber). Each path is like a comb filter and has transfer function $(1 - g_i z^{-N_i})$ where N_i is the length of the delay of the path in samples and g_i is the reflection coefficient of the wall for that reflection.

TABLE II

STAGE:	1	2	3	4	5
	(49.96,0.75)	(34.65,0.72)	(24.18,0.691)	(17.85,0.649)	(10.98,0.662)
				(18.01,0.646)	(11.45,0.650)
					(11.05,0.660)
					(10.82,0.666)
STAGE:	1	2	3	4	5
	(62.38,0.75)	(43.63,0.720)	(30.74,0.692)	(19.88,0.671)	(12.15,0.671)
				(19.49,0.676)	(12.93,0.651)
					(13.16,0.649)
					(13.55,0.638)

Delays and gains for two 5-stage, quadrasonic all-pass reverberators. Each unit reverberator is represented here by two numbers in parentheses. The first number is the delay of the unit in milliseconds, and the second number is the gain. The upper reverberator is designed for a total reverberation time of 1.2 seconds and the lower one is designed for 1.5 seconds. Each path from the first stage through the last stage in a particular channel has the same total reverberation time, although each path is different enough that the reverberation in each channel sounds uncorrelated with each other channel. The impulse response of the first path given here is shown in figure 29b.

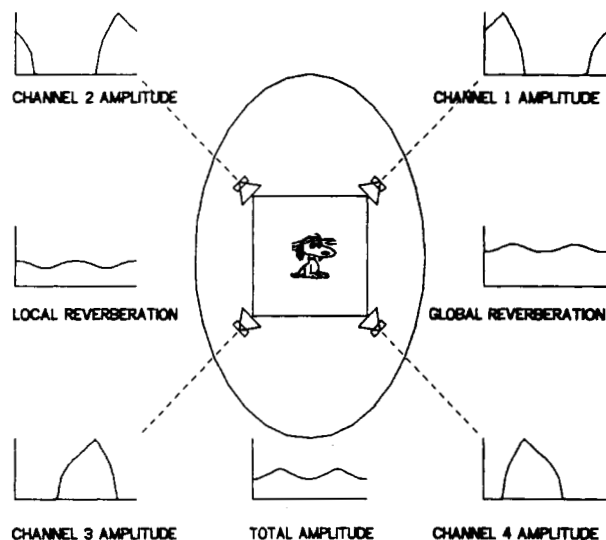


Fig. 32. Quadraphonic setup showing four speakers with listener in the center. The ellipse surrounding the speakers shows a hypothetical sound trajectory. It begins at the top, swings to the right and down, then back up to the left and stops at the top. The four graphs behind the speakers show the strength of the sound source in that speaker to produce the desired spatial illusion. In addition, there is an amount of reverberation that is distributed the same way the source is distributed, called "local" reverberation, and an amount of reverberation that is distributed equally to all four speakers. This is called "global" reverberation. As the sound moves away from the listener, the global reverberation is decreased and the local reverberation is increased. This helps simulate the fact that reflections of nearby sounds come from all directions whereas reflections of distant sounds originate predominately in the direction of the sound source. The lowest plot shows the total sound intensity as presented to the listener. The ellipse here has been approximated with 32 line segments, as have each of the 7 smaller graphs.

sensitive to time delay between the ears and have remarkable powers of localization when this cue is available (see, for instance, Leakey [32]). We can use this cue when we know that the listener will be wearing headphones, but for a concert hall or home stereo situation, the timing is completely scrambled by changing the position of the listener. Just moving one foot away from one speaker is enough to reverse the time cue. As long as we are committed to using speakers and audiences, time delay cannot be effectively used as a cue for sound localization.

There is another consideration that arises from the context of performance before an audience. One might ask why more than two speakers are necessary, seeing as how people only have two ears. Indeed, in a headphone situation, only two channels are necessary. All locational cues can be duplicated using stereo headphones. In a concert hall, however, more speakers are needed to assure that listeners that are not in the center of the audience still get some spatial effects. Although in theory, four speakers placed at the corners of a tetrahedron can duplicate exactly the sound pressure wave at a single point in the center of the tetrahedron (a fact that leads some engineers to conclude that any more speakers are unnecessary), this ignores the fact that not all the audience can sit at this point. When one considers this carefully, it is clear that additional (independent) channels can enhance the effect for those not sitting in the center of the speakers. This, then, is the motivation for using many channels. We have yet to determine the "optimum" price-performance tradeoffs for concert-hall work. As far as we know, no unified theory has been de-

veloped for analysing the effect of many channels on the spatial illusion for listeners not at the center of the listening space.

To return to the spatial location setup, Fig. 32 shows the four speakers and listener, but also shows the exact functions used to distribute the energy and the reverberation. The trajectory shown in this figure is an oval. The illusory location of the sound should start at the top, circle around to the right, then back up to the top. The plots behind each speaker show what the amplitude of the signal to that speaker should be. The plot in center bottom shows the total amplitude of the signal. There are two reverberation controls: local and global reverberation. Local reverberation is reverb that comes from the same place as the sound source itself. Global reverberation comes from all speakers evenly. To enhance the direction cue, we bring up the local reverberation when the sound is far away and the global reverberation when the sound is close.

THE ALL-DIGITAL RECORDING STUDIO

There are many functions performed in a modern recording studio. One popular method of producing commercial recordings is to "layer" up the piece one voice at a time. In this mode of operation, usually a tape recorder with many (for instance, 16) independent tracks is used. Several of the tracks are played through earphones for the performer so the new layer can be precisely synchronized with the other voices. To produce the final stereo tape, the multi-track recording must be "mixed down." This is where the loudness balance among the voices is achieved. The final "master" tape, then, is the weighted sum of the original tracks.

This mode of operation offers a great deal of flexibility in that the performer can repeat a particular section again and again until he gets it like he wants it. He need only preserve his best "takes" for the final mixdown.

Often other special effects are added in the mixdown, such as filtering or reverberation. Sometimes these effects require making yet another copy of the "master" recording.

There are several problems with producing recorded music this way. The first problem is that several copies of the recording have to be made. Each time a tape is copied, more noise is introduced into the recording. The two most common forms of noise in tape recordings are hiss (wide-band white noise) and dropout (irregularities in the oxide coating). Another problem is that the number of simultaneous layers that can be used before mixdown must occur is limited by the number of independent channels of the tape machine used. 16-track tape recorders are common in the industry these days, so the maximum number of independent layers is, in most cases, only 16.

Thus far we have only considered the recording process. There are also modification processes which stem from *musique concrète* practices. Some of the modifications that are commonly used are changes of speed, overlay (musical "collages"), filtering, and amplitude modulation. These techniques, limited mainly by the number and type of modifications available to the composer, are also limited by the amount of tape copying and overlaying that must be done to produce "thick" textures from many oftentimes fragmented parts.

The processes of recording, mixdown, and modification can all be aided by digital processing. Fig. 33 shows a diagram of the recording and playback setup. The recording studio itself

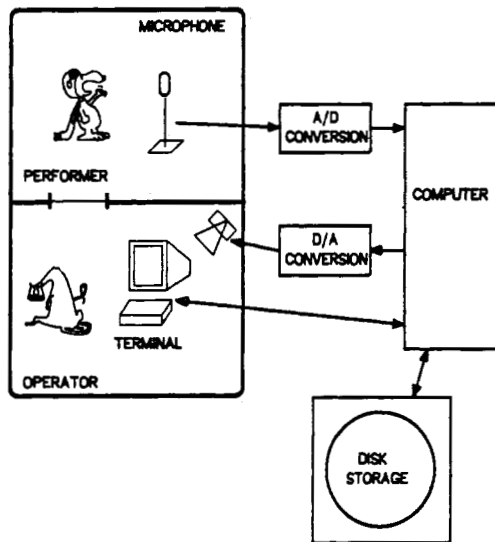


Fig. 33. Highly schematic representation of the disk-oriented all-digital recording studio. The performer and the operator are in adjacent acoustically treated rooms connected by a window. The microphone is connected directly to the analog-to-digital conversion equipment which is connected to the computer. The computer can store data on a large disk system. The digitized sound can be played back by conversion through a digital-to-analog conversion system and then played through speakers or earphones. The operator has a quiet computer terminal with which he directs the digitizing process. The advantage of this form of storage is that the only noise is the microphone pre-amplifier noise, the conversion noise (quantizing, clock jitter, etc), and noise from the computer electronics. There is no tape noise. In addition, the possibilities for long-term storage are much more favorable for digital recordings than analog recordings.

consists of two adjacent rooms, joined by a soundproof window. These rooms must be acoustically treated. In the recording room, the performer plays into a high-quality microphone. The signal from the microphone is amplified and transmitted differentially ("balanced") to the analog-to-digital (A/D) conversion equipment of the computer. Since computers are notoriously noisy, both electrically and acoustically, the computer itself must be physically isolated from the recording room. The conversion equipment produces a quantized sampled-data representation of the microphone signal which can then be stored on large high-speed storage devices, such as disks or tape. The sampled data can then, under program control, be retrieved from the storage medium, fed to a digital-to-analog (D/A) converter, sent differentially back to the recording studio, and be played either over speakers or earphones. The operator could be located in the room adjacent to the recording room with a quiet computer terminal. The operator would be responsible for starting and stopping conversion, and ordering and labeling of the takes.

There are commercially available A/D and D/A converters with up to 16 bits of precision. With this accuracy, up to 96 dB of dynamic range can be captured. In practice, however, dynamic range is usually limited by residual noise in the system, such as hiss from the microphone amplifier, hum pickup in the wires back to the computer, and noise from the digital electronics of the computer itself.

One of the first prototype all-digital computer-based recording studios was realized at Stanford last year under the motivation of L. Rush in the Department of Music [61]. Such a facility has been available at the University of Utah for some years now. There is also now a commercial all-digital recording studio available (Easton [20]). At Stanford, the only equipment in the studio was a quiet terminal, a microphone

```

Command: RECORD
File name: TRY1
Writing file TRY1
New file. Type maximum length (in seconds): 20
Type C.R. when ready ("A" to abort):
Was this a take? Y

```

```

Command: HISTOGRAM
File name:
Reading file TRY1

```

RANGE	FRACTION
0	1
1	0
2	0
3	0
4	0
5	0
6	0
7	0

```

Command: RECORD
File name:
Writing file TRY1
File already exists. Use its length? Y
Type C.R. when ready ("A" to abort):
Was this a take? Y

```

```

Command: HISTOGRAM
File name:
Reading file TRY1

```

RANGE	FRACTION
0	.45
1	.28
2	.17
3	.09
4	.01
5	0
6	0
7	0

```

Command: PLAY
File to play:
Playing file TRY1
Type number of times to play: 2
Type number of times to play:

```

Command:

Fig. 34. Sample exchange with recording studio program. The responses typed by the user are shown in *italics*. The program starts by asking for a command. The user types *RECORD*, meaning we wish to digitize some sound. The program asks for a file name. This is a 1 to 6 character identifier for the take. The program also asks for the maximum length of the file in seconds. When the program is ready to go, it waits for a carriage-return from the operator for acknowledgement (and to synchronize the program and the performer). This is all done on-line in interactive mode. The operator then asks for an amplitude histogram of the file he just digitized. Typing carriage return when the program asks for a file name gets the last file written. The histogram is printed out as the fraction of time the signal spent in each of 8 divisions of the amplitude range. For instance, range 0 was amplitudes 0 to 0.125. Range 7 was amplitudes 0.875 to 1.0. This histogram indicated that the signal spent all of its time in the first range. This is too small for most purposes, so the gain of the microphone amplifier was increased (not shown here). The take was done over again. The next histogram indicates a reasonable spread of amplitudes, so that recording was saved.

(and its line amplifier), and a power amplifier and speaker. The audio was delivered to the A/D and from the D/A by balanced shielded pair cable. A program was set up to do the sorts of things done in recording sessions: start a take, abort or confirm a take, keep copies of different takes around. The program also gives maximum amplitudes encountered in a given take as well as an amplitude histogram. Fig. 34 shows some sample output of the program which includes the starting of a take, terminating a take early, retaking, confirming a take, and doing a histogram for a volume level check. At Stanford, we did not have the ability to play and record at the same time. This limits the utility of the system because the performer could not hear previously recorded voices to synchronize with. For the particular piece for which the record-

ing was being done, synchronization was not necessary, but in general the ability to play and record at the same time is essential.

Once the takes are digitized, another set of programs are used to break up the files into useful segments and reassemble them in various ways. The "digital mixdown" has the advantage that there is no limit to the number of overlays, because no perceivable accumulation of error need occur. By delaying and overlaying many different copies of the same sound, often a composite sound can be produced that sounds like a large number of performers. This overlay technique can be used to produce "choral" effects from a small number of digitized segments. The timing of various voices being overlayed can be adjusted interactively using graphical display or audio presentation, and can be adjusted down to the sample level.

"Microsurgery" can be done on the signals to eliminate certain kinds of errors (like coughing between phrases on a single voice). This kind of surgery can, of course, be done with standard audio tape. The advantage of using the computer, however, is many fold: as we mentioned, timing can be adjusted to an extremely precise level, down to the individual sample if need be. Also, the process of locating beginnings and endings of segments of sound can be largely automated. Graphical aids can be utilized to show boundaries of sounds to help the composer decide where the best "splicing points" are.

Through the use of digital interpolation and decimation (Crooke and Craig [17], Schafer and Rabiner [65], Bellanger *et al.* [7], Oetken *et al.* [53], Shively [69], Crochiere and Rabiner [16]) tuning can be adjusted to a very fine level. The procedure is to adjust the effective sampling rate of the signal, so that when it is played at the original sampling rate, the pitch (and duration) of the signal is changed. This is done through the use of a digital interpolator and decimator. Fig. 35(a) shows a block diagram of this process. The first step is an interpolation of $1:M$, which effectively raises the sampling rate by a factor of M by filling in the $M-1$ points between samples of the original waveform. This is best accomplished by low-pass filtering using a linear-phase finite impulse response filter (Schafer and Rabiner [65]). The result of this interpolation is a signal at an intermediate sampling rate M times the original rate. The next step is to reduce the sampling rate by a factor of N by digital decimation, which implies first low-pass filtering to eliminate any objectionable aliasing, then taking every N th point of the thus filtered waveform. As shown in Fig. 35(b), the low-pass filtering operations of interpolation and decimation can be combined into a single filtering operation. A great reduction in computation time can be made by noticing that the filter need only be applied every N th point of the intermediate signal. Further reductions can be made by doing the process in stages (Crochiere and Rabiner [16]). This accomplishes a change of pitch by a factor of N/M where N and M are both integers. Thus the amount we wish to adjust the fundamental frequency by must be approximated by a rational number. This is not a significant restriction in practice because quite large ratios can be used, such as 2000/2001, which gives about 0.05-percent change. This is about one hundredth of a half step in pitch. Since we only have to evaluate the low-pass filter every N th point of the intermediate signal, if N and M are quite close, then the limiting factor is just the size of the filter used. The length of the impulse response of the filter must be approximately $K * \max(M, N)$ (Hermann *et al.* [27]), where K is chosen to give sufficient stop-band rejection and narrow enough transi-

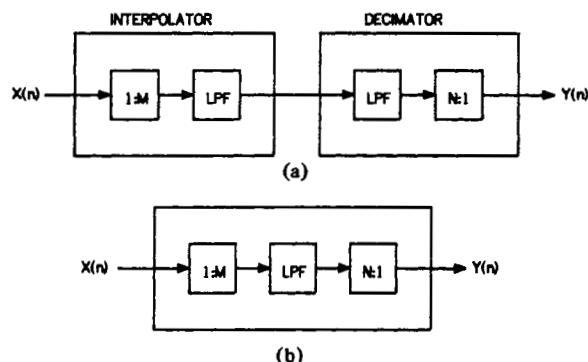


Fig. 35. (a) Block diagram of an interpolator-decimator that can be used to effectively speed up or slow down a digitized segment of sound by a factor of N/M . The process is to first interpolate the waveform by a factor of M , low-pass filtering to eliminate the higher copies of the spectrum, then decimate the waveform by first low-pass filtering to eliminate objectionable aliasing then picking one out of every N points as our output waveform. (b) We see that the low-pass filtering operation can be combined into one filter.

tion width for clean alias-free sound. On a large-scale computing system with large amounts of central memory, quite large filters can be accommodated, thus allowing very precise control of retuning ratios. We have found that $K = 8$ is a reasonable compromise between desire for high stopband rejection and desire for short compute time. This number was determined empirically by listening tests. This filter length seems to work well for a variety of sounds. For sounds with a great deal of high-frequency energy, however, longer filters may be needed. The ability to retune a segment is very important. It allows us, for instance, to correct a mistuned note in the middle of an otherwise perfect take.

We can apply any of the techniques discussed thus far in this paper to natural sounds which have been digitized. For instance, we can add concert-hall reverberation to any digitized sound. We can try out different reverberators on a sound to see which one gives the best results for that particular sound. We can also "fly" the sound in an illusory trajectory around the room. This would be done, as we mentioned before, by distributing the energy of the signal to four or more speakers to simulate the azimuth cue, and controlling the ratio of direct sound to reverberant sound to simulate the distance cue.

The sound could also be modified by analyzing the sound and then modifying the analysis data before resynthesizing the sound. The previously described analysis techniques can be used for this purpose when applicable (as we mentioned before, the additive synthesis analysis technique is only meaningful for isolated tones of nearly constant frequency, and the subtractive synthesis analysis techniques can not model the excitation function accurately). We can also combine analysis data from different sounds, as was done by Petersen [55]–[57] using the subtractive synthesis model.

Currently, the limiting factor in the use of digital media for recording sound is the extreme bulk involved. For instance, if we attempted to record 16 independent tracks of wide-bandwidth sound, that would require 16 channels of 40 000 samples (or more) per second, each sample requiring 16 bits. This adds up to about 10 million bits per second. For comparison, the IBM 3330 model II disk storage system has a maximum transfer rate of somewhat more than half that. At that rate, an entire 3330 disk pack would be completely filled in about 150 s. If all that was required was to record linearly without random access or overlay, we could record onto high-density magnetic tape. If we wish to do random access or

overlay, however, some form of random access storage, such as disk, is required.

Although duplicating the functions of a commercial 16-track studio directly seems to be unfeasible right now, there are some things we can do which make digital recording useful today. As we mentioned, one thing we can do is record linearly. Another thing is that we can play one or two voices back using a high-speed disk while simultaneously recording one or two voices, although this has not been done that we know of to date. A third prospect is that of modification of the digitized sounds. As mentioned before, there is a wider range of modifications that can be done using digital processing than is available to the analog studio. Each of these modifications can be made using as many bits in the computation as are necessary to assure no perceivable distortion occurs.

Unfortunately, as far as we know, no comprehensive study has been made of how many bits are needed for various types of modifications to preserve enough quality for musical purposes.

SIGNAL PROCESSING HARDWARE FOR MUSIC

To determine how much compute power is necessary for real-time operation, we must first define the program material and the goals. For instance, if we set as our goal the synthesis of an entire 120-piece orchestra, we cannot hope to locate the funding to build such a synthesizer. On the other hand, if we are content with one to four voices of FM, we can realize this on a general-purpose minicomputer. A reasonable compromise goal is to ask for 12 voices (10 fingers and 2 feet) of reasonably complex instruments, such as additive synthesis or subtractive synthesis instruments. If we choose additive synthesis and require 16 harmonics for each voice, each with piecewise-linear amplitude and frequency functions, we can estimate the number of multiplies, adds, and sinusoids that must be synthesized each sample. Each sinusoid has a frequency, a frequency change rate, an angle, an amplitude, and an amplitude change rate. To get the next sample of that sinusoid, we must update the frequency by the frequency change rate, update the angle by the frequency, update the amplitude by the amplitude change rate, look up the sine of the angle in read-only memory, and multiply the value of the sinusoid by the amplitude. This, then, is the output sample. This process is diagrammed in Fig. 36. All of this must take place in one sample time. If we assume a 25-kHz sampling rate, this gives us 40 μ s for each sample. Let us not forget that this process must be done for each of the 16 harmonics in each voice, and for each of the 12 voices. In addition, another add must be done for each sinusoid to sum them all together to provide the final output waveform. This totals about 800 adds, and 200 sine table lookups and 200 multiplies. At a typical rate of 40 μ s per sample, this says that the effective add rate must be about 20 adds per microsecond, 5 table lookups per μ s, and 5 multiplies per μ s. Although no definitive study of the number of bits needed for these computations has been done, it is fairly well agreed that at least 16 bits must be carried along at each stage of the computation. This computation rate is enormous. At the time of this writing, such rates can only be achieved through use of extremely expensive large-scale general purpose computers, or through special-purpose hardware synthesizers.

Since all of the preceding methodology involves quite a bit of computation, it is natural to ask what kind of help can we get from special-purpose hardware to achieve real-time oper-

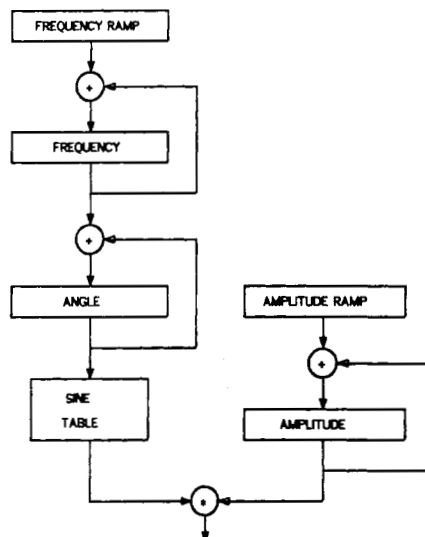


Fig. 36. Computational flow for a single sinusoidal oscillator with one piece of a piecewise linear amplitude control and piecewise linear frequency control. This algorithm requires three additions, one multiplication, and one table lookup.

ation. There are, of course, many commercial signal processors that are capable of great speed. Unfortunately, commercial signal processors have generally required the use of difficult and idiosyncratic programming techniques. Easily programmable signal processors have generally been incapable of providing the high instruction execution rates needed for music. Furthermore, costs and architectures of these processors have often limited their usefulness in parallel processing applications. We might then ask if there are special properties of music synthesis that could influence the design of such a device. Here, we can take a lesson from the electronic musicians. Most analog synthesizers are set up in terms of special functional modules. Each module has control ports and data ports (sometimes these ports are compatible so that control and data voltages may be intermixed). The modules are interconnected with patch cords to produce a signal processing complex for a particular sound. The interconnection is called the "patch" and defines the processing sequence. One thing to notice from this description is that the above process is a "stream" procedure. The interconnections (the "patch") change slowly, but the data being processed changes quite rapidly. The "patches" must be very flexible and easily changeable to promote experimentation and diversity.

As an example, let us look at the modules that are available in a common electronic music synthesizer catalog:

Voltage-Controlled Oscillator—Usually produces sine, square, and/or triangle waveforms. The frequency is proportional to the exponential of the input voltage, that is, the output frequency doubles for every linear increase in control voltage.

Voltage-Controlled Filter—Hi-pass, low-pass, or bandpass functions are usually offered. Again, the frequency is set by the exponential of the control voltage.

Voltage-Controlled Amplifier—Essentially a 2-quadrant multiplier. Used for applying slowly varying functions to signals, such as amplitude envelopes.

Ring Modulator—A 4-quadrant multiplier, used for amplitude modulation.

Transient generators—These come in many forms, but generally allow use 2 to 4 pieces of a piecewise-exponential function to be synthesized. These are usually used as amplitude or frequency envelopes.

Noise source—Produces wide-band white noise.

There are, of course, many other modules that are available, but these give us an example of the kinds of tools we use with analog synthesis. We can take a source, either an oscillator, a noise source, or natural sound, and modify it by controlling its amplitude with a multiplier of some kind, filtering it, modulating it with another signal, or process it with any of a number of other modules that are available.

One natural extension, then, would be to design a device with a large number of somewhat special purpose functional modules with some kind of interconnection matrix or memory for setting up patches. These devices have the advantage that a composer with extensive analog synthesizer experience can easily "program" such a device. He only has to become accustomed to the particular functional modules the device has.

One might argue that this could be done through software on any general-purpose signal processing device. One could set up a set of programs that realize the various functional modules, and the composer could piece them together as he pleases. This is true in a technical sense, but has shortcomings in a practical sense. Since the signal processing devices are highly pipelined and quite complex to program, their execution speed depends strongly on how clever the program is. They are usually set up for "block" operations, where a large amount of data is processed by a single algorithm, then the algorithm is changed and another large block of data is processed. This is different from the "stream" approach, where one sample of many different data streams is processed, then the next sample of many different data streams is processed. In certain applications, for instance when feedback is used, the block method cannot be used, and the entire computational algorithm, feedback and all, must be merged into a single program. This implies that extensive reprogramming is necessary every time an algorithm is attempted that cannot be done in blocks.

Let us discuss two of the many possible designs for special purpose digital music synthesizers. We will choose for discussion a commercial device, designed by Peter Samson of Systems Concepts of San Francisco, and a research device designed by F. R. Moore at Stanford. These particular devices were chosen based on the author's extensive knowledge of these devices and not with any commitment that these represent necessarily the ultimate design choices for such devices. This is a new field and the future certainly holds a wide range of innovations. There is quite a bit of room for clever design here.

These digital synthesizers would best be utilized in a configuration such as that realized in the GROOVE system at Bell Laboratories [43]. A CPU with a large disk would be interfaced to the synthesizer. To this computer would be connected interactive graphical input and output devices, and a variety of knobs, buttons, and switches, possibly including one or more organ keyboards. The program is then designed to take "instrument" descriptions from the user and connect the various input devices to the instantiation and parameter control of the instruments in any manner desired that is consistent with real-time operation. One nice thing about this setup is that all the user's interactions can be very precisely

recorded. This "action record" contains all the performance nuance that can be so valuable in producing a natural sounding piece of music. Moreover, this action record can be edited, modified, corrected, or processed in any number of ways. Experience with the GROOVE system shows that graphical devices are invaluable for "seeing" what is going on and especially for editing control functions.

At some point, the CPU cannot supply enough compute power to maintain real time operation. In this case, the voices must be layered up one at a time, supplying all the interaction one voice at a time. This is still a perfectly reasonable way to work and still offers such tremendous advantage to "off-line" composition as to be a most important goal. This mode of operation makes it important that it be possible to transmit sampled data between the device and the CPU so that this layering can easily be done.

THE MOORE DIGITAL SYNTHESIZER

This digital synthesizer is being designed at Stanford University by F. R. Moore as part of his doctoral research in the Department of Electrical Engineering. The design was based on the idea that since we do not know exactly what processing techniques will be required in the future, the functional modules should be pluggable in somewhat the same way as analog synthesizers are. This necessitates separating the design into two parts: the control unit which serves as an interconnection matrix and delivers sequencing and data routing information to the functional modules, and of course the modules themselves. One set of modules might be digital filters, another set might be sinusoidal oscillators, another set might be general-purpose multipliers, and so on.

This device has up to eight pluggable modules, each one of which is time-division multiplexed 32 ways, to make an effective total of 256 functional units. Data is routed by two 20-bit busses. In each of the 256 time slices of a sample, one functional unit puts its output datum on the data-out bus and reads its next input datum from the data-in bus. The control unit stores the output datum in the signal memory. The input datum also came from signal memory, but from a location whose address comes from a read-address memory. This memory does nothing but store read addresses which are used to control which word in the signal memory gets to which functional unit. The computer loads the "patch" into this read-address memory and this controls the interconnections. All interconnections are done through the signal memory.

We should say something about modularity here. What we mean here by a pluggable module is one or two 9 by 11 inch wire-wrap boards. This is clearly more difficult to replace than a module that interconnects with only a few wires, such as modern stereo equipment. The point of modularity in this device is that the various functions are partitioned so that all the operations for a given processing element are on the same board. If this partitioning is not designed in from the beginning, functions can get spread over many boards, making changes in the processing elements quite difficult.

Each functional unit also receives a 12-bit control byte that comes from a control memory that is filled by the computer. These control bytes can be thought of as operation codes for the functional units. On each sample, we circulate through all the functional units in order. A new datum is read from and written into the signal memory every 120 ns. Each module performs one complete computation every 8 steps (the mod-

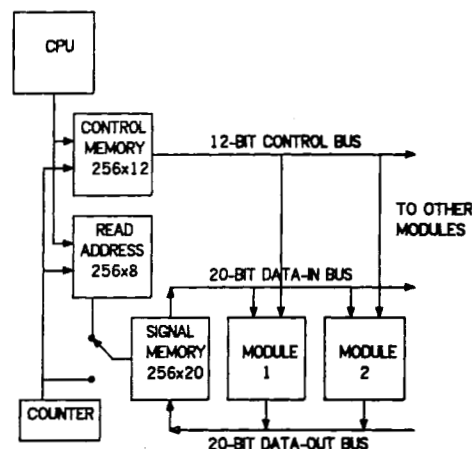


Fig. 37. Block diagram of Moore's digital synthesizer. There are up to eight function modules. Each module is time-division multiplexed 32 ways. Thus there are a total of 256 functional units. The modules are pluggable so that the device can be entirely reconfigured by plugging in different modules. During a particular time division, a functional unit will receive a datum from the data-in bus, a control byte from the control bus, and will gate a datum onto the data-out bus. The signal memory stores the contents of the data-out bus and reads its contents onto the data-in bus. The write addresses are just cyclic and correspond to the number of the functional unit (3 bits for module numbers, 5 bits for slice within that module). The read addresses are set by the CPU and are stored in the read address memory. The D/A converters themselves can be functional modules, as can the CPU.

ules are serviced in order) or about 1 microsecond. This means that all 256 functional units are referenced roughly every 32 μ s, yielding a sampling rate of approximately 32 kHz.

Functional modules typically have more inputs than outputs. A module can easily plug into, for instance, two slots and thus get two data from the data-in bus and can then generate up to two output data. Let us look at some specific modules to see how all this works.

One functional module would certainly be an oscillator module. This would synthesize up to 32 oscillators at once. It might take two inputs. For instance, it could take in two frequencies, add them internally, and produce a sine, square, frequency modulated, or bandlimited pulse waveform at that composite frequency. This module would then have two inputs and one output.

One could also imagine a general-purpose envelope generator (amplitude modulator) which takes a control signal in one input and a signal to be modulated in the other. In the trivial case, the module just multiplies its two inputs and delivers that to the output. In the more complex case, the control input could be, for instance, integrated to produce piecewise-linear envelopes. We could also put memory in the module itself to allow it to store time constants for attacks and decays.

Fig. 37 shows a block diagram of the device. We see the signal memory data output supplies the data-in bus for the modules. Likewise, the data-out bus from the modules goes to the write data input of the signal memory. The addressing of the signal memory is sequential for writing, but is set from the read-address memory for reading. Each functional unit then deposits in a dedicated slot but reads from a settable slot.

Since most units have more inputs than outputs, the extra data-out bus slots could be used by the CPU to supply more data to the signal memory. Again, the slots used by the CPU would be pluggable, in keeping with the design philosophy of the device.

Thus we see that this device can perform a great deal of computation all in parallel. It can be easily "patched," any functional unit can "talk" to any other functional unit, and the computer can be made to provide as much data as is necessary for the device. The pluggable nature of the modules assures that as new techniques are developed, new modules could be built to perform the required computations.

One further option might be to have a general-purpose 20-bit microprocessor as one of the modules. This would allow experimentation with algorithms without having to physically construct new modules. Unfortunately, to get the required throughput in a highly programmable device would require state-of-the-art hardware which is at this time not cost-effective. We can, however, look forward to ever-decreasing hardware costs which at some point may make programmable devices attractive again.

THE SYSTEMS CONCEPTS DIGITAL SYNTHESIZER

Peter Samson of Systems Concepts, Inc., of San Francisco has designed and built a large-scale digital synthesizer. The Systems Concepts device has a fixed menu of functional modules: there are up to 256 "generators" and 128 "modifiers." They are interconnected by a RAM called "sum" memory. There is, in addition, an associated memory of up to 64K 20-bit words which can be used as up to 32 tapped delay lines or table lookup. Fig. 38 shows a simplified block diagram of the device. Each generator reads one datum from sum memory and outputs one datum. Each modifier can read two data and write one datum. These functional modules are time-division multiplexed. The device operates on a 195-ns "tick." Each generator takes one tick and each modifier takes two ticks. Ticks on which computations take place are called processing ticks. There are also update ticks which are used to change the controlling parameters of any of the functional modules. The total of processing ticks plus update ticks (plus 9 for pipelining) determines the overall sample rate. If all the generators are to be used, the maximum sampling rate would be somewhat less than 20 kHz. If only half the generators are needed, the resulting sampling rate could be as high as 40 kHz. These figures would be somewhat lower if many update ticks were needed for a given task.

Each generator and modifier has a control memory that determines its function and holds its running values and parameters. This control memory can be written by the CPU, and this is how the device is operated.

A generator can have several different functions, depending on its "mode," which is a parameter in its control memory. A generator can produce nonbandlimited square waves, pulse trains, sawtooth signals as well as pure sinusoids and band-limited pulse trains (Winham and Steiglitz [75]). For all of these modes, the input datum from sum memory serves as a frequency modulation control so that any of these waveforms can be frequency modulated by other functional modules. Inside the generator there is a provision for linearly sweeping the frequency in addition to the external frequency modulation input. There is also amplitude control of any of the generated waveforms. The amplitude may be changed either linearly or exponentially. The result of this power is that control functions (like amplitudes and frequencies of sinusoids with time) in piecewise-linear form can be used directly by the hardware. This corresponds nicely to our analysis form for additive synthesis.

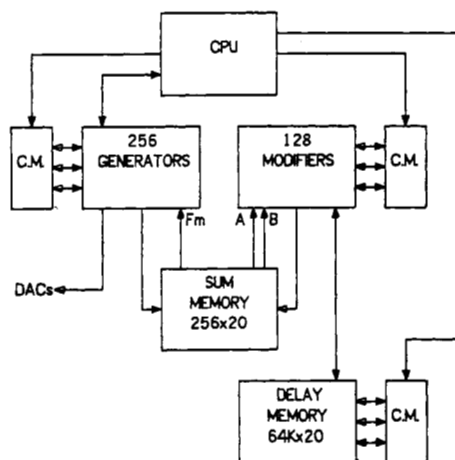


Fig. 38. Block diagram of the Systems Concepts Digital Synthesizer, designed by Samson. Each generator can synthesize a sinusoid, band-limited pulse train, nonlimited square wave, sawtooth, or pulse train. The amplitude of the waveform can also be modulated by a piece of a piecewise linear or exponential envelope. Each modifier can do two poles or zeros (second-order digital filter section, either numerator or denominator), amplitude modulation, 4-quadrant multiplication, white noise generation, max and min functions, zero-cross pulsing, and in conjunction with the delay memory, can perform the operations of a unit reverberator. The delay memory can be partitioned in to up to 32 independent sections. Each section can be a tapped delay line, for use in unit reverberators, or can serve as a table for generalized function approximation (like square root, reciprocal, arctangent, etc). A generator can also be a data channel which can transfer sample-time data to or from the CPU or the D/A converters. The sum memory serves as an interconnection matrix for the functional modules. The modules are controlled through their control memories (CM) which can be set by the CPU.

A generator can also be used as a data channel to pass sampled data between the sum memory and the CPU. This allows mixing of precomputed or prerecorded sounds with freshly generated or modified sounds. The output can also be stored by the CPU. This means that the device will never be outgrown, because the data can always be shuffled back to the CPU for anything the device cannot do. We consider this provision one of the most important features of the device. Too many synthesizer designs do not allow this most simple process and thus have "hard" limitations.

A modifier can be used to synthesize a second-order section of a digital filter (either two poles or two zeros but not both), amplitude modulation, 4-quadrant multiplication, max and min functions, zero crossing pulser, random noise (by the linear congruential method of Knuth [31]) and several others. When used in conjunction with the delay memory, unit reverberators or table lookup can be realized. The table lookup feature we consider quite important because this allows one to do general purpose functional approximation for getting functions that are not hard-wired into the device, such as division (reciprocal table), square roots (for energy normalization), arctangents, and numerous others. This is the "escape" that allows quite a bit of generality, at the cost of only having 32 such functions that can be accessed at once.

Most of the data paths through the machine are 20-bit fixed-point paths. The sine table in the generators is effectively 8192 locations long. The multiplies in the modifiers are 20×20 multiplies. Both sum memory and delay memory are 20-bit fixed-point memories.

The name of the sum memory comes from the fact that it is cleared at the beginning of a pass through all the functional

units (one sample) and each write into this memory adds into the location rather than replaces. Each modifier has the option of adding into memory or replacing, but the generators only add into sum memory. There are actually two sum memories, one for this pass and one for last pass. Reading from sum memory usually comes from the last pass, so there is a one sample delay in passing data from one functional module to the other, but ordering of the modules is not critical when using this mode. Since the modifiers can read from either sum memory (this pass or last pass), they can be cascaded with no delay. When using this feature, care must be given to the ordering of the units. The programmer must be assured that the previous unit has deposited its output in the memory before the next unit is to read it. Since the machine is extensively pipelined, the data is deposited several ticks after it is read.

From these descriptions, it should be clear that these devices are good for other things than straightforward music or speech synthesis. They are moderately general signal processing devices. Although the addressing is not versatile enough to implement, for instance, the fast Fourier transform algorithm, these devices can do things like autocorrelations, matrix dot products, windowing, covariances, general purpose digital filtering and many more. Through the use of stored functions, equations (2) through (7) could be computed by these devices in real time. This would allow us to do certain kinds of analysis and synthesis entirely in real time.

SUMMARY AND CONCLUSIONS

The composition and production of high quality digitally synthesized music involves almost every aspect of signal processing. Likewise, almost every aspect of computer music must be treated with the respect due a sampled data system with attention to the special problems of sampled data at every stage of processing. We have discussed some of the goals of digital synthesis, some of the techniques found useful, and some hardware architectures that offer real-time possibilities.

The composer desires control of every aspect of sound: time, frequency, loudness, timbre, and spacial environment. We wish to be able to use natural sounds as points of departure for exploring the little understood world of timbre. We wish to study perception to better understand how we hear and how our pieces will be perceived. We wish to study signal processing to better understand what analytic and synthetic tools are available, as well as how to avoid making troublesome mistakes due to violating the rules of sampled data systems.

In this paper, we have discussed several methods of analyzing, synthesizing, and processing musical sound. We have divided synthesis techniques into *additive* methods, *subtractive* methods, and *nonlinear* methods. For additive and subtractive techniques there exist analysis techniques which can directly supply synthesis parameters from the sounds of existing music instruments. The analysis for the additive technique described herein produces synthetic sound with remarkable similarity to the original sound, but is only useful for isolated tones of nearly constant frequency. The linear predictor analysis for subtractive synthesis is not constrained to isolated tones of constant frequency, but has not to date produced the great fidelity of reproduction that the additive-based analysis has. The nonlinear techniques have no corresponding analysis technique and thus must be developed by intuition and trial-and-

error in an interactive manner. Despite this seeming handicap, many interesting sounds have been created using summation formula synthesis, especially frequency modulation. Frequency modulation synthesis is the simplest technique discussed and is most appropriate for systems with limited resources. It is also the most amenable to real-time operation.

Several ways of processing sound were also discussed. These included modification from analysis data and modification through spatial manipulation. Using the linear predictor, which is analysis-based, we can modify the spectra of sounds to give vowel quality to nonspeech sounds. Using spatial manipulation, we can give the illusion of sounds moving through the space around the listener, as well as the illusion of the listening environment.

Through all-digital recording and mixing, many of the problems with modern recording studios can be eliminated. These problems include tape hiss and irregularities of the oxide. These problems limit the amount of studio processing that can be done. In an all-digital system, we can always use sufficient word length to assure that the arithmetic itself does not introduce distortion into the signal.

Digital signal processing offers many advantages for the analysis, synthesis, and processing of sound. Only the computation speed and the expense are limiting factors now. Digital technology still seems to be advancing in an exponential manner, so that eventually speed and expense will catch up with the needs of the composer, the musician, the music world.

ACKNOWLEDGMENT

It seems impossible here to credit all the people that made this research, and even this paper possible. We first must thank our co-workers at the Stanford Center for Computer Research in Music and Acoustics. These include (in alphabetical order) J. Chowning, J. Grey, and L. Rush. Without the tireless contributions of these people, much of this research would never have been done. Most of the figures in this paper and much of the research is attributable to these people. Special thanks must go to J. Chowning, without whose constant inspiration, encouragement, guidance and support, both intellectual and emotional, the Stanford computer music group would never have existed. Thanks go to J. Makhoul whose stimulating discussions and diligent proofreading helped make this work what it is. We must also thank L. Rabiner, without whose inspiration and proofreading this manuscript would never have come to be. Further thanks to G. Loy, C. Dodge, T. Petersen, B. Vercoe, P. Wood, R. Poor, J. Gordon, R. Moore, and P. Samson. And last but not most important of all, to the National Science Foundation and the National Endowment for the Arts, without whose support little of this would have been possible.

REFERENCES

- [1] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions*. New York: Dover, 1965.
- [2] J. H. Appleton, "A simple musical language for the Dartmouth digital synthesizer," presented at 2nd Music Computation Conf., Urbana, ILL, Nov. 7-9, 1975.
- [3] B. S. Atal and M. R. Schroeder, "Adaptive predictive coding of speech signals," *Bell Syst. Tech. J.*, vol. 49, pp. 1973-1986, 1970.
- [4] B. S. Atal and S. L. Hanauer, "Speech analysis and synthesis by linear prediction of the speech wave," *J. Acoust. Soc. Amer.*, vol. 50, pp. 637-655, Feb. 1971.
- [5] B. S. Atal and L. R. Rabiner, "A pattern recognition approach to voiced-unvoiced-silence classification with applications to speech recognition," *IEEE Trans. Acoust., Speech, and Signal Processing*, vol. ASSP-24, pp. 201-211, June 1976.
- [6] G. von Békésy, *Experiments in Hearing*. New York: McGraw-Hill, 1960.
- [7] M. G. Bellanger, J. L. Daguet, and G. P. Lepagnol, "Interpolation, extrapolation, and reduction of computation speed in digital filters," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-22, pp. 231-235, Aug. 1974.
- [8] A. H. Benade, *Horns, Strings, and Harmony*. Garden City, NY: Doubleday, 1960.
- [9] —, "The physics of brasses," *Scientific American*, vol. 229, no. 1, pp. 24-35, July 1973.
- [10] —, *Fundamentals of Musical Acoustics*. New York: Oxford Univ. Press, 1976.
- [11] R. B. Blackman and J. W. Tukey, *The Measurement of Power Spectra*. New York: Dover, 1959.
- [12] J. M. Chowning, "The simulation of moving sound sources," *J. Audio Eng. Soc.*, vol. 19, no. 1, 1971.
- [13] —, "The synthesis of complex audio spectra by means of frequency modulation," *J. Audio Eng. Soc.*, vol. 21, no. 7, pp. 526-534, Sept. 1973.
- [14] J. M. Chowning, J. M. Grey, J. A. Moorer, and L. Rush, "Computer simulation of music instrument tones in reverberant spaces," Dep. Music, Stanford Univ., Stanford, CA, Tech. Rep. STAN-M-1, 1975.
- [15] M. S. Corington, "Variation of bandwidth with modulation index in frequency modulation," *Proc. IRE*, vol. 35, pp. 1013-1020, Oct. 1947.
- [16] R. E. Crochiere and L. R. Rabiner, "Optimum FIR digital filter implementations for decimation, interpolation, and narrow-band filtering," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-23, pp. 444-456, Oct. 1975.
- [17] A. W. Crooke and J. W. Craig, "Digital filters for sample-rate reduction," *IEEE Trans. Audio Electroacoust.*, vol. AU-20, pp. 308-315, Oct. 1972.
- [18] M. G. Crosby, "Carrier and side-frequency relations with multi-tone frequency or phase modulation," *RCA Rev.*, vol. 3, pp. 103-106, 1938.
- [19] C. Dodge, "Synthetic Speech Music," Composer's Recordings, Inc., New York, CRI-SD-348, 1975 (disk).
- [20] R. Easton, "Soundstream: the first digital studio," *Recording Engineer/Producer*, vol. 7, no. 2, pp. 57-61, Apr. 1976.
- [21] M. B. Gardner, "Proximity image effect in sound localization," *J. Acoust. Soc. Amer.*, vol. 43, no. 1, p. 163, 1968.
- [22] B. Gold and L. R. Rabiner, "Parallel processing techniques for estimating pitch periods of speech in the time domain," *J. Acoust. Soc. Amer.*, vol. 46, no. 2, pp. 442-448, Aug. 1969.
- [23] R. W. Gosper, "Acceleration of series," *Artificial Intelligence*, MIT, Memo 304, Mar. 1974.
- [24] —, "A calculus of series rearrangements," in *Proc. Symp. New Directions and Recent Results in Algorithms and Complexity* (Carnegie-Mellon Univ., Pittsburgh, PA), 1976.
- [25] J. M. Grey, "An exploration of musical timbre," Ph.D. dissertation, Dep. Psychology, Stanford Univ., Stanford, CA, 1975. Distributed through the Department of Music, Stanford University, as Rep. STAN-M-2.
- [26] E. R. Hansen, *A Table of Series and Products*. Englewood Cliffs, NJ: Prentice-Hall, 1975.
- [27] O. Hermann, L. R. Rabiner, and D. S. K. Chan, "Practical design rules for optimum finite impulse response low-pass digital filters," *Bell Syst. Tech. J.*, vol. 52, no. 6, pp. 769-799, July-Aug. 1973.
- [28] L. H. Holway, Jr., and J. A. Mullen, "Exact and approximate spectra for trapezoidal FM," *Proc. IEEE*, vol. 64, pp. 380-381, Mar. 1976.
- [29] H. S. Howe, Jr., *Electronic Music Synthesis*. New York: W. W. Norton and Co., Inc., 1975.
- [30] F. Itakura and S. Saito, "Analysis synthesis telephony based upon the maximum likelihood method," *Rep. 6th Int. Congr. Acoust.*, Y. Konasi, Ed., Tokyo, Japan, Rep. C-5-5, Aug. 21-28, 1968.
- [31] D. E. Knuth, *The Art of Computer Programming—Volume II*. Reading, MA: Addison-Wesley, 1969.
- [32] D. M. Leakey, "Some measurements on the effects of inter-channel intensity and time differences in two channel sound systems," *J. Acoust. Soc. Amer.*, vol. 31, no. 7, pp. 977-985, July 1969.
- [33] J. Makhoul and J. J. Wolf, "Linear prediction and the spectral analysis of speech," Bolt, Beranek and Newman, Inc., Cambridge, MA, Report 2304, Aug. 1972.
- [34] J. Makhoul, R. Viswanathan, L. Cosell, and W. Russell, "Natural communication with computers, final report—Volume II, Speech Compression Research at BBN," Bolt, Beranek, and Newman Rep. 2976, Dec. 1974.
- [35] J. Makhoul, "Linear prediction: A tutorial review," *Proc. IEEE*, vol. 63, pp. 561-580, Apr. 1975.
- [36] —, "New lattice methods for linear prediction," in *Conf. Rec. 1976 IEEE Int. Conf. Acoustics, Speech, and Signal Processing*, pp. 462-465, Apr. 12-14, 1976, Catalog Number 76CH1067-8 ASSP.
- [37] J. D. Markel, "Digital inverse filtering—A new tool for formant trajectory estimation," *IEEE Trans. Audio Electroacoust.*, vol. AU-20, pp. 129-137, June 1972.

- [38] —, "The SIFT algorithm for fundamental frequency estimation," *IEEE Trans. Audio Electroacoust.*, vol. AU-20, pp. 367-377, Dec. 1972.
- [39] J. D. Markel, A. H. Gray, Jr., and H. Wakita, "Linear prediction of speech—Theory and practice," SCRL Monograph 10, Sept. 1973.
- [40] J. D. Markel and A. H. Gray, Jr., "A linear prediction vocoder simulation based upon the autocorrelation method," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-22, pp. 124-134, Apr. 1974.
- [41] —, *Linear Prediction of Speech*. New York: Springer-Verlag, 1976.
- [42] M. V. Mathews, *The Technology of Computer Music*. Boston, MA: MIT Press, 1969.
- [43] M. V. Mathews and F. R. Moore, "GROOVE—A program to compose, store, and edit functions of time," *Commun. ACM*, vol. 13, pp. 715-721, Dec. 1970.
- [44] M. V. Mathews and J. Kohout, "Electronic simulation of violin," *J. Acoust. Soc. Amer.*, vol. 53, pp. 1620-1626, 1973.
- [45] N. J. Miller, "Pitch detection by data reduction," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-23, pp. 72-79, Feb. 1975.
- [46] D. Morrill, "Trumpet algorithms for computer composition," *Comput. Music J.*, vol. 1, no. 1, pp. 46-52, Feb. 1977.
- [47] —, "Suite for computer and trumpet," available from the composer, Dep. Music, Colgate Univ., Colgate, NY, 1974.
- [48] J. A. Moorer, "The optimum comb method of pitch period analysis of continuous digitized speech," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-22, pp. 330-338, Oct. 1974.
- [49] —, "On the segmentation and analysis of continuous musical sound by digital computer," Ph.D. dissertation, Dep. Computer Science, Stanford Univ., Stanford, CA, May 1975. Distributed through the Department of Music, Stanford Univ., Rep. STAN-M-3.
- [50] —, "The synthesis of complex audio spectra by means of discrete summation formulae," *J. Audio Eng. Soc.*, vol. 24, no. 9, pp. 717-727, Nov. 1976.
- [51] —, "The use of the phase vocoder in computer music applications," *Audio Eng. Soc. Preprint 1146 (E-1)*, 55th Conv., Oct. 29-Nov. 1, 1976.
- [52] A. M. Noll, "Cepstrum pitch determination," *J. Acoust. Soc. Amer.*, vol. 41, pp. 293-309, Feb. 1967.
- [53] G. Oetken, T. W. Parks, and H. W. Schüssler, "New results in the design of digital interpolators," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-23, pp. 301-309, June 1975.
- [54] A. V. Oppenheim, "Speech analysis-synthesis system based on homomorphic filtering," *J. Acoust. Soc. Amer.*, vol. 45, no. 2, pp. 458-465, 1969.
- [55] T. L. Petersen, "Vocal tract modulation of instrumental sounds by digital filtering," presented at the Music Computation Conf. II, School of Music, Univ. Illinois, Urbana-Champaign, Nov. 7-9, 1975.
- [56] —, "Dynamic sound processing," in *Proc. 1976 ACM Computer Science Conf.* (Anaheim, CA), Feb. 10-12, 1976.
- [57] —, "Analysis-synthesis as a tool for creating new families of sound," presented at the 54th Conv. Audio Engineering Society (Los Angeles, CA), May 4-7, 1976.
- [58] M. R. Portnoff, "Implementation of the digital phase vocoder using the fast Fourier transform," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-24, pp. 243-248, June 1976.
- [59] L. R. Rabiner, "Techniques for designing finite impulse-response digital filters," *IEEE Trans. Commun. Technol.*, vol. COM-19, pp. 188-195, Apr. 1971.
- [60] J. C. Risset and M. V. Mathews, "Analysis of musical-instrument tones," *Physics Today*, vol. 22, pp. 23-30, Feb. 1969.
- [61] L. Rush, D. G. Loy, and J. A. Moorer, "The Stanford digital recording studio," *in preparation*.
- [62] S. Saito and F. Itakura, "The theoretical consideration of statistically optimum methods for speech spectral density" (in Japanese), Elec. Commun. Lab., N.T.T., Tokyo, Japan, Rep. 3107, Dec. 20, 1966.
- [63] S. E. Saunders, "Real-time FM digital music synthesis," in *Proc. Music Computation Conf. 11* (Urbana, IL), Nov. 1975.
- [64] R. W. Schafer, "Echo removal by discrete generalized linear filtering," Ph.D. dissertation, MIT, Cambridge, MA, MIT RLE Tech. Rep. 466, Feb. 28, 1969.
- [65] R. W. Schafer and L. R. Rabiner, "A digital signal processing approach to interpolation," *Proc. IEEE*, vol. 61, pp. 692-702, June 1973.
- [66] M. R. Schroeder, "Improved quasi-stereophony and colorless artificial reverberation," *J. Acoust. Soc. Amer.*, vol. 33, p. 1061, 1961.
- [67] M. R. Schroeder and B. F. Logan, "Colorless artificial reverberation," *J. Audio Eng. Soc.*, vol. 9, no. 3, p. 192, July 1961.
- [68] M. R. Schroeder, "Natural sounding artificial reverberation," *J. Audio Eng. Soc.*, vol. 10, p. 219, 1962.
- [69] R. R. Shively, "On multistage finite impulse response (FIR) filters with decimation," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-23, pp. 353-357, Aug. 1975.
- [70] A. W. Slawson, "Vowel quality and musical timbre as functions of spectrum envelope and fundamental frequency," *J. Acoust. Soc. Amer.*, vol. 43, no. 1, pp. 87-101, 1968.
- [71] M. M. Sondhi, "New methods of pitch extraction," *IEEE Trans. Audio Electroacoust.*, vol. AU-16, pp. 262-266, June 1968.
- [72] J. Stirling, *The Differential Method*, London, 1749.
- [73] B. Vercoe, "The MUSIC 360 language for digital sound synthesis," in *Proc. American Society of University Composers*, vol. 6, 1971.
- [74] —, "The relevance of interactive computing to creativity in music and research," presented at 2nd Music Computation Conf., Urbana, IL, Nov. 7-9, 1975.
- [75] G. Winham and K. Steiglitz, "Input generators for digital sound synthesis," *J. Acoust. Soc. Amer.*, vol. 47, no. 2 (part 2), pp. 665-666, 1970.

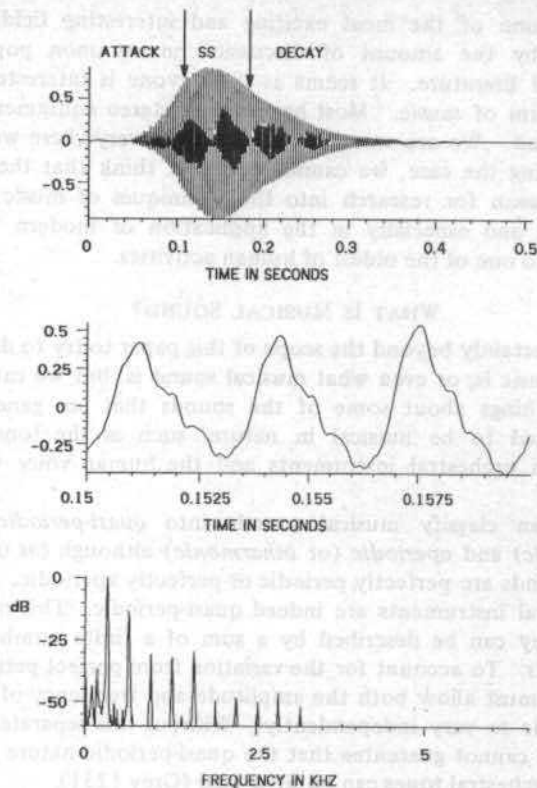


Fig. 1. Example of an isolated cello tone. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform containing about three periods. The lower plot is the discrete Fourier transform of an 80-ms segment in the middle of the note. The upper plot is divided, somewhat arbitrarily, into three segments: the *attack* portion, the *steady-state* portion, and the *decay* portion.

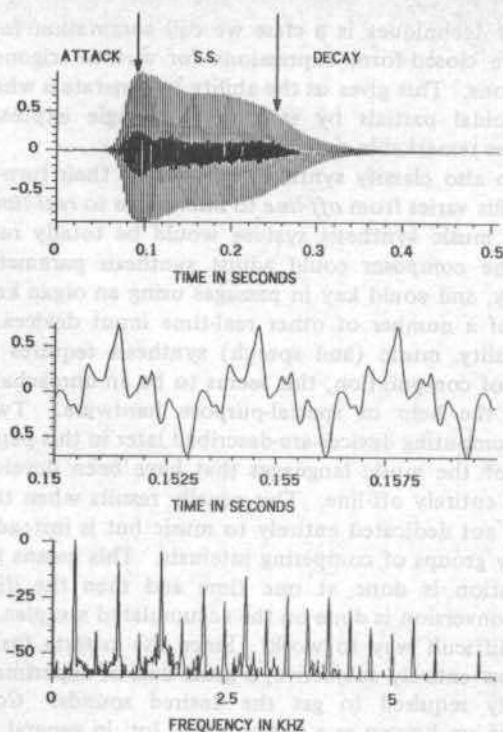


Fig. 2. Example of an isolated clarinet tone. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform containing about three periods. The lower plot is the discrete Fourier transform of an 80-ms segment in the middle of the note. The upper plot is divided, again somewhat arbitrarily, into three segments: the *attack* portion, the *steady-state* portion, and the *decay* portion. In comparing these plots with the previous plots for the cello tone, we can see the following differences: 1) The clarinet has more energy in the odd harmonics than the even harmonics, 2) The clarinet has a larger number of significant harmonics than the cello, and 3) The attack and decay of the clarinet are much faster than those of the cello.

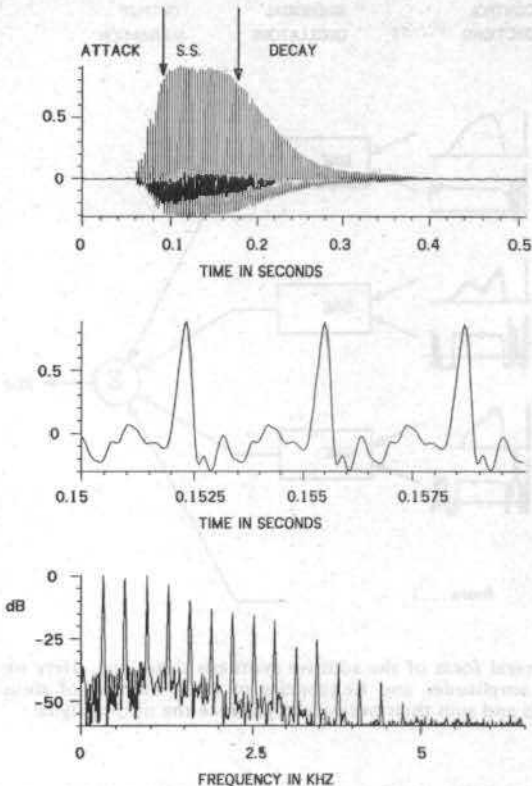


Fig. 3. Example of an isolated trumpet tone. The upper plot shows the waveform of the entire note. The second plot shows a segment of the same waveform containing about three periods. The lower plot is the discrete Fourier transform of an 80-ms segment in the middle of the note. The upper plot is divided into three segments: the *attack* portion, the *steady-state* portion, and the *decay* portion.

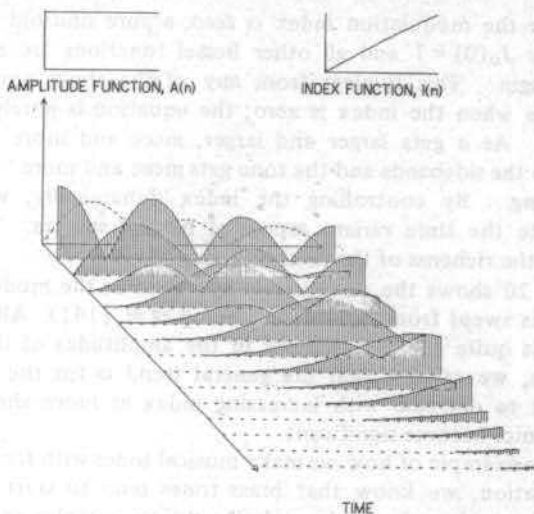


Fig. 20. Perspective plot of the spectrum of a frequency modulation signal with $f_m = f_c$ as the modulation index is swept from 0 to 8. The small plots above the perspective plot show the amplitude (on the left) and the modulation index (on the right). For this plot, the amplitude was held constant. The axis coming out of the page is frequency, starting at zero at the origin. The dotted line at frequency = 0 is a dc term that always accompanies the FM signal.

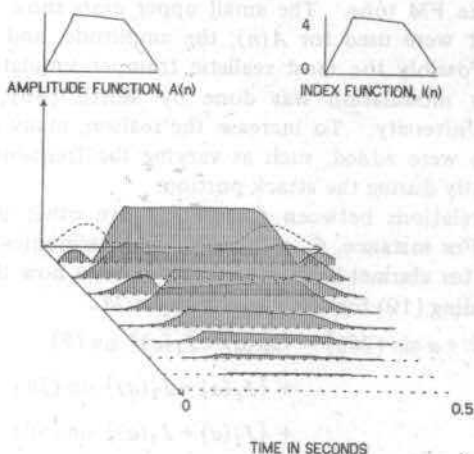


Fig. 21. Perspective plot of the spectrum of a frequency modulation signal of a brass-like tone. In this case, f_c was set equal to f_m . The index function is the same shape as the amplitude function. The index starts at 0 and goes to a maximum of 4, then decays back to zero. The entire tone takes about 0.5 s. We can see how the fundamental comes in first, followed by higher and higher harmonics. This mimics the general spectral trend of the trumpet [9], [60]. These controlling parameters are also applicable to other summation formulae. For instance, either equation (15) or equation (16) could be used to produce a brass-like tone with this same amplitude function and same index function. Each of the tones will be different, but all will have a brass-like character. Likewise, any of the equations (19) through (22) could be used with the number of harmonics (N) set to about 8 (in the finite cases, equations (19) and (21)) and the maximum index scaled to be about 0.82 (rather than 4 as above).